DSA 502 - Final Project

AI-Powered Personalized Nutrition Recommendation System

Project Overview

Maintaining a healthy diet requires understanding individual calorie needs and selecting foods that align with those nutritional requirements. However, many people struggle to estimate how many calories they need daily and which foods best support their health goals.

This project aims to develop an **AI-powered personalized nutrition recommendation system** that combines machine learning, nutrition datasets, and data visualization to help users make better dietary decisions.

The system will include three main components:

1. Calorie Prediction Model

A machine learning model will be built to **predict daily calorie expenditure** using physiological and activity-related features such as age, gender, height, weight, heart rate, and exercise duration.

2. Food Recommendation System

Using a nutrition dataset containing macronutrient information for different foods, the system will recommend foods that align with the predicted calorie needs and provide balanced nutrition.

3. Interactive Nutrition Dashboard

An interactive dashboard will visualize nutritional information, food comparisons, and recommended foods to help users better understand dietary options.

Research Questions

1. Can machine learning accurately predict calorie expenditure using physiological and exercise-related features?

2. How can nutrition datasets be used to recommend foods that match calorie requirements and nutritional balance?
3. How can visual dashboards help users better interpret nutrition data and recommendations?

Hypotheses

- Physiological and activity-related variables can be used to reasonably predict calorie expenditure.
- Nutrition datasets can be used to identify foods that match calorie and macronutrient requirements.
- Data visualization can improve the interpretability of nutrition data and recommendations.

Project Goals

The main objectives of this project are:

- Predict daily calorie needs using machine learning models
- Recommend nutritionally appropriate foods based on predicted calorie needs
- Build an interactive dashboard for nutrition analysis and recommendations

---

Target Variable

Calories

The machine learning model will predict the number of calories burned based on physiological and exercise-related features.

Data Card

Dataset 1: Exercise and Calories Dataset

This dataset contains physiological and exercise-related information used to predict calorie expenditure. Each row represents an individual exercise session with associated body characteristics and activity metrics.

Source

Kaggle dataset: *Calories Burned Prediction Dataset*

Key Variables

- **User_ID** – Unique identifier for each record
- **Gender** – Biological gender of the individual
- **Age** – Age in years
- **Height** – Height in centimeters
- **Weight** – Weight in kilograms
- **Duration** – Duration of exercise in minutes
- **Heart_Rate** – Heart rate during exercise
- **Body_Temp** – Body temperature during exercise
- **Calories** – Calories burned (target variable)

Purpose

This dataset will be used to train a **machine learning regression model** that predicts calorie expenditure based on physiological and exercise-related features.

Dataset 2: Food Nutrition Dataset

This dataset contains nutritional information for a wide range of foods, including macronutrients and micronutrients.

Source

Kaggle dataset: *Food Nutrition Dataset*

Key Variables

- **Food** – Name of the food item
- **Calories** – Energy value of the food
- **Protein** – Protein content (grams)
- **Fat** – Fat content (grams)
- **Carbohydrates** – Carbohydrate content (grams)
- **Fiber** – Fiber content (grams)
- **Sugar** – Sugar content (grams)
- **Sodium** – Sodium content (milligrams)

Purpose

This dataset will be used to:

- Recommend foods that match predicted calorie requirements
- Perform nutritional comparisons across foods
- Build visualizations for the nutrition dashboard

Dataset 3: Recipes Dataset

This dataset contains recipe information such as recipe names, ingredients, and preparation steps.

Purpose

This dataset may be used to extend the recommendation system from individual foods to **complete meal recommendations**.

Risks and Limitations

- Calorie prediction models may not capture individual metabolic differences.
- Nutrition datasets may contain estimated nutritional values rather than exact laboratory measurements.
- Dietary preferences and medical conditions are not included in the dataset.
- Some foods may appear multiple times or contain incomplete nutrition information.

Ethical Considerations

- Nutrition recommendations should not replace professional medical or dietary advice.
- The system should be used as a **decision-support tool**, not as a medical prescription.
- Cultural dietary differences and personal dietary restrictions should be considered in real-world applications.

Data Loading

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Folder path
path = r"C:\University Course\DSA 502\AI-Powered Personalized Nutrition Recommendat

# ----------------------------------------
# Load exercise and calories datasets
```

```python
# ----------------------------------------
exercise = pd.read_excel(path + r"\exercise.xlsx")
calories = pd.read_excel(path + r"\calories.xlsx")

# Merge datasets
df = pd.merge(exercise, calories, on="User_ID")

# ----------------------------------------
# Load food nutrition datasets
# ----------------------------------------
food1 = pd.read_excel(path + r"\FOOD-DATA-GROUP1.xlsx")
food2 = pd.read_excel(path + r"\FOOD-DATA-GROUP2.xlsx")
food3 = pd.read_excel(path + r"\FOOD-DATA-GROUP3.xlsx")
food4 = pd.read_excel(path + r"\FOOD-DATA-GROUP4.xlsx")
food5 = pd.read_excel(path + r"\FOOD-DATA-GROUP5.xlsx")

# Combine food datasets
food_df = pd.concat([food1, food2, food3, food4, food5], ignore_index=True)

# ----------------------------------------
# Load recipes dataset
# ----------------------------------------
recipes_df = pd.read_excel(path + r"\Receipes from around the world.xlsx")

# ----------------------------------------
# Preview datasets
# ----------------------------------------
print("Merged calorie dataset:")
display(df.head())

print("Food nutrition dataset:")
display(food_df.head())

print("Recipes dataset:")
display(recipes_df.head())

# ----------------------------------------
# Dataset shapes
# ----------------------------------------
print("Merged dataset shape:", df.shape)
print("Food dataset shape:", food_df.shape)
print("Recipes dataset shape:", recipes_df.shape)
```

```
Merged calorie dataset:
```

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190	94	29	105	40.8	231
1	14861698	female	20	166	60	14	94	40.3	66
2	11179863	male	69	179	79	5	88	38.7	26
3	16180408	female	34	179	71	13	100	40.5	71
4	17771927	female	27	154	58	10	81	39.8	35

```
Food nutrition dataset:
```

	Unnamed: 0.1	Unnamed: 0	food	Caloric Value	Fat	Saturated Fats	Monounsaturated Fats	Polyunsatur
0	0	0	cream cheese	51	5.0	2.9	1.3	(
1	1	1	neufchatel cheese	215	19.4	10.9	4.9	(
2	2	2	requeijao cremoso light catupiry	49	3.6	2.3	0.9	(
3	3	3	ricotta cheese	30	2.0	1.3	0.5	(
4	4	4	cream cheese low fat	30	2.3	1.4	0.6	(

5 rows × 37 columns

```
Recipes dataset:
```

	recipe_name	cuisine	ingredients	cooking_time_minutes	prep_time_minutes	servings	ca
0	Spicy Kimchi Fried Rice	Korean	['Cooked rice', 'Kimchi', 'Gochujang', 'Soy sa...	25	15	2.0	
1	Classic Margherita Pizza	Italian	['Pizza dough', 'Tomato sauce', 'Fresh mozzare...	20	30	4.0	
2	Coconut Chickpea Curry	Indian	['Chickpeas', 'Coconut milk', 'Onion', 'Ginger...	35	15	4.0	
3	Pad See Ew with Tofu	Thai	['Wide rice noodles', 'Tofu', 'Chinese broccol...	20	10	2.0	
4	Beef Bourguignon	French	['Beef chuck', 'Red wine', 'Bacon', 'Onion', '...	150	30	6.0	

```
Merged dataset shape: (15000, 9)
Food dataset shape: (2395, 37)
Recipes dataset shape: (161, 8)
```

Data Audit

In [9]:

```
# ====================================
# Data Audit
# ====================================

# ----------------------------
# 1. Dataset Shapes
# ----------------------------
print("===== DATASET SHAPES =====")
print("Merged calorie dataset shape:", df.shape)
print("Food nutrition dataset shape:", food_df.shape)
print("Recipes dataset shape:", recipes_df.shape)

# ----------------------------
# 2. Column Names
```

```
# ----------------------------
print("\n===== COLUMN NAMES =====")
print("\nMerged calorie dataset columns:")
print(df.columns.tolist())

print("\nFood nutrition dataset columns:")
print(food_df.columns.tolist())

print("\nRecipes dataset columns:")
print(recipes_df.columns.tolist())

# ----------------------------
# 3. Data Types
# ----------------------------
print("\n===== DATA TYPES =====")
print("\nMerged calorie dataset dtypes:")
print(df.dtypes)

print("\nFood nutrition dataset dtypes:")
print(food_df.dtypes)

print("\nRecipes dataset dtypes:")
print(recipes_df.dtypes)

# ----------------------------
# 4. Missing Values
# ----------------------------
print("\n===== MISSING VALUES =====")
print("\nMerged calorie dataset missing values:")
print(df.isnull().sum())

print("\nFood nutrition dataset missing values:")
print(food_df.isnull().sum())

print("\nRecipes dataset missing values:")
print(recipes_df.isnull().sum())

# ----------------------------
# 5. Duplicate Rows
# ----------------------------
print("\n===== DUPLICATE ROWS =====")
print("Duplicate rows in merged calorie dataset:", df.duplicated().sum())
print("Duplicate rows in food nutrition dataset:", food_df.duplicated().sum())
print("Duplicate rows in recipes dataset:", recipes_df.duplicated().sum())

# ----------------------------
# 6. Basic Summary Statistics
# ----------------------------
print("\n===== SUMMARY STATISTICS: MERGED CALORIE DATASET =====")
display(df.describe(include="all"))

print("\n===== SUMMARY STATISTICS: FOOD NUTRITION DATASET =====")
display(food_df.describe(include="all"))

print("\n===== SUMMARY STATISTICS: RECIPES DATASET =====")
display(recipes_df.describe(include="all"))
```

```
# ----------------------------
# 7. Preview of Each Dataset
# ----------------------------
print("\n===== FIRST 5 ROWS =====")
print("\nMerged calorie dataset preview:")
display(df.head())

print("\nFood nutrition dataset preview:")
display(food_df.head())

print("\nRecipes dataset preview:")
display(recipes_df.head())

# ----------------------------
# 8. Unique Value Check for Key Categorical Columns
# ----------------------------
print("\n===== UNIQUE VALUE CHECK =====")

if "Gender" in df.columns:
    print("\nUnique values in Gender:")
    print(df["Gender"].value_counts(dropna=False))

if "cuisine" in recipes_df.columns:
    print("\nTop cuisines in recipes dataset:")
    print(recipes_df["cuisine"].value_counts(dropna=False).head(15))

if "dietary_restrictions" in recipes_df.columns:
    print("\nTop dietary restriction patterns:")
    print(recipes_df["dietary_restrictions"].value_counts(dropna=False).head(15))

# ----------------------------
# 9. Check for Unnecessary Columns in Food Dataset
# ----------------------------
print("\n===== POSSIBLE UNNECESSARY COLUMNS IN FOOD DATASET =====")
possible_extra_cols = [col for col in food_df.columns if "Unnamed" in col]
print(possible_extra_cols)

# ----------------------------
# 10. Check Unique IDs
# ----------------------------
print("\n===== ID CHECK =====")
if "User_ID" in df.columns:
    print("Unique User_ID count in merged calorie dataset:", df["User_ID"].nunique(
    print("Total rows in merged calorie dataset:", len(df))

# ----------------------------
# 11. Check Numeric Column Ranges
# ----------------------------
print("\n===== NUMERIC COLUMN RANGES =====")

print("\nMerged calorie dataset numeric ranges:")
for col in df.select_dtypes(include=["number"]).columns:
    print(f"{col}: min = {df[col].min()}, max = {df[col].max()}")

print("\nFood nutrition dataset numeric ranges:")
```

```python
for col in food_df.select_dtypes(include=["number"]).columns:
    print(f"{col}: min = {food_df[col].min()}, max = {food_df[col].max()}")

print("\nRecipes dataset numeric ranges:")
for col in recipes_df.select_dtypes(include=["number"]).columns:
    print(f"{col}: min = {recipes_df[col].min()}, max = {recipes_df[col].max()}")
```

```
===== DATASET SHAPES =====
Merged calorie dataset shape: (15000, 9)
Food nutrition dataset shape: (2395, 37)
Recipes dataset shape: (161, 8)

===== COLUMN NAMES =====

Merged calorie dataset columns:
['User_ID', 'Gender', 'Age', 'Height', 'Weight', 'Duration', 'Heart_Rate', 'Body_Tem
p', 'Calories']

Food nutrition dataset columns:
['Unnamed: 0.1', 'Unnamed: 0', 'food', 'Caloric Value', 'Fat', 'Saturated Fats', 'Mo
nounsaturated Fats', 'Polyunsaturated Fats', 'Carbohydrates', 'Sugars', 'Protein',
'Dietary Fiber', 'Cholesterol', 'Sodium', 'Water', 'Vitamin A', 'Vitamin B1', 'Vitam
in B11', 'Vitamin B12', 'Vitamin B2', 'Vitamin B3', 'Vitamin B5', 'Vitamin B6', 'Vit
amin C', 'Vitamin D', 'Vitamin E', 'Vitamin K', 'Calcium', 'Copper', 'Iron', 'Magnes
ium', 'Manganese', 'Phosphorus', 'Potassium', 'Selenium', 'Zinc', 'Nutrition Densit
y']

Recipes dataset columns:
['recipe_name', 'cuisine', 'ingredients', 'cooking_time_minutes', 'prep_time_minute
s', 'servings', 'calories_per_serving', 'dietary_restrictions']

===== DATA TYPES =====

Merged calorie dataset dtypes:
User_ID         int64
Gender         object
Age             int64
Height          int64
Weight          int64
Duration        int64
Heart_Rate      int64
Body_Temp     float64
Calories        int64
dtype: object

Food nutrition dataset dtypes:
Unnamed: 0.1              int64
Unnamed: 0                int64
food                     object
Caloric Value             int64
Fat                     float64
Saturated Fats          float64
Monounsaturated Fats    float64
Polyunsaturated Fats    float64
Carbohydrates           float64
Sugars                  float64
Protein                 float64
Dietary Fiber           float64
Cholesterol             float64
Sodium                  float64
Water                   float64
Vitamin A               float64
Vitamin B1              float64
```

```
Vitamin B11             float64
Vitamin B12             float64
Vitamin B2              float64
Vitamin B3              float64
Vitamin B5              float64
Vitamin B6              float64
Vitamin C               float64
Vitamin D               float64
Vitamin E               float64
Vitamin K               float64
Calcium                 float64
Copper                  float64
Iron                    float64
Magnesium               float64
Manganese               float64
Phosphorus              float64
Potassium               float64
Selenium                float64
Zinc                    float64
Nutrition Density       float64
dtype: object

Recipes dataset dtypes:
recipe_name              object
cuisine                  object
ingredients              object
cooking_time_minutes      int64
prep_time_minutes         int64
servings                float64
calories_per_serving    float64
dietary_restrictions     object
dtype: object

===== MISSING VALUES =====

Merged calorie dataset missing values:
User_ID       0
Gender        0
Age           0
Height        0
Weight        0
Duration      0
Heart_Rate    0
Body_Temp     0
Calories      0
dtype: int64

Food nutrition dataset missing values:
Unnamed: 0.1            0
Unnamed: 0              0
food                    0
Caloric Value           0
Fat                     0
Saturated Fats          0
Monounsaturated Fats    0
Polyunsaturated Fats    0
```

```
Carbohydrates           0
Sugars                  0
Protein                 0
Dietary Fiber           0
Cholesterol             0
Sodium                  0
Water                   0
Vitamin A               0
Vitamin B1              0
Vitamin B11             0
Vitamin B12             0
Vitamin B2              0
Vitamin B3              0
Vitamin B5              0
Vitamin B6              0
Vitamin C               0
Vitamin D               0
Vitamin E               0
Vitamin K               0
Calcium                 0
Copper                  0
Iron                    0
Magnesium               0
Manganese               0
Phosphorus              0
Potassium               0
Selenium                0
Zinc                    0
Nutrition Density       0
dtype: int64

Recipes dataset missing values:
recipe_name             0
cuisine                 0
ingredients             0
cooking_time_minutes    0
prep_time_minutes       0
servings                4
calories_per_serving    5
dietary_restrictions    0
dtype: int64

===== DUPLICATE ROWS =====
Duplicate rows in merged calorie dataset: 0
Duplicate rows in food nutrition dataset: 0
Duplicate rows in recipes dataset: 0

===== SUMMARY STATISTICS: MERGED CALORIE DATASET =====
```

	User_ID	Gender	Age	Height	Weight	Duration	He
count	1.500000e+04	15000	15000.000000	15000.000000	15000.000000	15000.000000	1500
unique	NaN	2	NaN	NaN	NaN	NaN	
top	NaN	female	NaN	NaN	NaN	NaN	
freq	NaN	7553	NaN	NaN	NaN	NaN	
mean	1.497736e+07	NaN	42.789800	174.465133	74.966867	15.530600	9
std	2.872851e+06	NaN	16.980264	14.258114	15.035657	8.319203	
min	1.000116e+07	NaN	20.000000	123.000000	36.000000	1.000000	6
25%	1.247419e+07	NaN	28.000000	164.000000	63.000000	8.000000	8
50%	1.499728e+07	NaN	39.000000	175.000000	74.000000	16.000000	9
75%	1.744928e+07	NaN	56.000000	185.000000	87.000000	23.000000	10
max	1.999965e+07	NaN	79.000000	222.000000	132.000000	30.000000	12

```
===== SUMMARY STATISTICS: FOOD NUTRITION DATASET =====
```

	Unnamed: 0.1	Unnamed: 0	food	Caloric Value	Fat	Saturated Fats	Monounsat
count	2395.000000	2395.000000	2395	2395.000000	2395.000000	2395.000000	2395.
unique	NaN	NaN	2395	NaN	NaN	NaN	
top	NaN	NaN	cream cheese	NaN	NaN	NaN	
freq	NaN	NaN	1	NaN	NaN	NaN	
mean	272.258038	272.258038	NaN	223.769520	10.176276	3.924917	4.
std	182.889214	182.889214	NaN	384.728244	29.008915	19.502262	12.
min	0.000000	0.000000	NaN	0.000000	0.000000	0.000000	0.
25%	119.000000	119.000000	NaN	44.500000	0.300000	0.064000	0.
50%	241.000000	241.000000	NaN	117.000000	2.100000	0.500000	0.
75%	414.500000	414.500000	NaN	258.000000	9.400000	2.700000	3.
max	721.000000	721.000000	NaN	6077.000000	550.700000	672.000000	291.

11 rows × 37 columns

```
===== SUMMARY STATISTICS: RECIPES DATASET =====
```

	recipe_name	cuisine	ingredients	cooking_time_minutes	prep_time_minutes	s
count	161	161	161	161.000000	161.000000	157
unique	161	24	161	NaN	NaN	
top	Spicy Kimchi Fried Rice	Argentinian	['Cooked rice', 'Kimchi', 'Gochujang', 'Soy sa...	NaN	NaN	
freq	1	12	1	NaN	NaN	
mean	NaN	NaN	NaN	62.614907	22.608696	5
std	NaN	NaN	NaN	76.166517	10.957973	3
min	NaN	NaN	NaN	5.000000	2.000000	1
25%	NaN	NaN	NaN	30.000000	15.000000	4
50%	NaN	NaN	NaN	45.000000	20.000000	4
75%	NaN	NaN	NaN	68.000000	30.000000	6
max	NaN	NaN	NaN	720.000000	60.000000	24

```
===== FIRST 5 ROWS =====

Merged calorie dataset preview:
```

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190	94	29	105	40.8	231
1	14861698	female	20	166	60	14	94	40.3	66
2	11179863	male	69	179	79	5	88	38.7	26
3	16180408	female	34	179	71	13	100	40.5	71
4	17771927	female	27	154	58	10	81	39.8	35

```
Food nutrition dataset preview:
```

	Unnamed: 0.1	Unnamed: 0	food	Caloric Value	Fat	Saturated Fats	Monounsaturated Fats	Polyunsatur
0	0	0	cream cheese	51	5.0	2.9	1.3	(
1	1	1	neufchatel cheese	215	19.4	10.9	4.9	(
2	2	2	requeijao cremoso light catupiry	49	3.6	2.3	0.9	(
3	3	3	ricotta cheese	30	2.0	1.3	0.5	(
4	4	4	cream cheese low fat	30	2.3	1.4	0.6	(

5 rows × 37 columns

```
Recipes dataset preview:
```

	recipe_name	cuisine	ingredients	cooking_time_minutes	prep_time_minutes	servings	ca
0	Spicy Kimchi Fried Rice	Korean	['Cooked rice', 'Kimchi', 'Gochujang', 'Soy sa...	25	15	2.0	
1	Classic Margherita Pizza	Italian	['Pizza dough', 'Tomato sauce', 'Fresh mozzare...	20	30	4.0	
2	Coconut Chickpea Curry	Indian	['Chickpeas', 'Coconut milk', 'Onion', 'Ginger...	35	15	4.0	
3	Pad See Ew with Tofu	Thai	['Wide rice noodles', 'Tofu', 'Chinese broccol...	20	10	2.0	
4	Beef Bourguignon	French	['Beef chuck', 'Red wine', 'Bacon', 'Onion', '...	150	30	6.0	

```
===== UNIQUE VALUE CHECK =====

Unique values in Gender:
Gender
female    7553
male      7447
Name: count, dtype: int64

Top cuisines in recipes dataset:
cuisine
Argentinian    12
Swedish        10
Japanese        9
Nigerian        9
Korean          8
Peruvian        8
Ethiopian       8
Australian      8
Thai            8
Mexican         7
French          7
Polynesian      7
Spanish         7
Brazilian       7
Greek           6
Name: count, dtype: int64

Top dietary restriction patterns:
dietary_restrictions
['nan']                                                   60
['vegetarian']                                            29
['gluten-free', 'dairy-free']                             28
['vegan', 'vegetarian', 'gluten-free', 'dairy-free']       8
['dairy-free']                                             8
['vegan']                                                  7
['vegetarian', 'gluten-free']                              5
['vegan', 'vegetarian']                                    3
['vegan', 'vegetarian', 'dairy-free']                      3
['vegan', 'vegetarian', 'gluten-free']                     2
['vegetarian', 'gluten-free', 'dairy-free']                2
['nut-free']                                               2
['gluten-free (depending on filling)']                     1
['dairy-free, gluten-free']                                1
['gluten-free, nut-free']                                  1
Name: count, dtype: int64

===== POSSIBLE UNNECESSARY COLUMNS IN FOOD DATASET =====
['Unnamed: 0.1', 'Unnamed: 0']

===== ID CHECK =====
Unique User_ID count in merged calorie dataset: 15000
Total rows in merged calorie dataset: 15000

===== NUMERIC COLUMN RANGES =====

Merged calorie dataset numeric ranges:
```

```
User_ID: min = 10001159, max = 19999647
Age: min = 20, max = 79
Height: min = 123, max = 222
Weight: min = 36, max = 132
Duration: min = 1, max = 30
Heart_Rate: min = 67, max = 128
Body_Temp: min = 37.1, max = 41.5
Calories: min = 1, max = 314

Food nutrition dataset numeric ranges:
Unnamed: 0.1: min = 0, max = 721
Unnamed: 0: min = 0, max = 721
Caloric Value: min = 0, max = 6077
Fat: min = 0.0, max = 550.7
Saturated Fats: min = 0.0, max = 672.0
Monounsaturated Fats: min = 0.0, max = 291.1
Polyunsaturated Fats: min = 0.0, max = 188.0
Carbohydrates: min = 0.0, max = 390.2
Sugars: min = 0.0, max = 291.5
Protein: min = 0.0, max = 560.3
Dietary Fiber: min = 0.0, max = 76.5
Cholesterol: min = 0.0, max = 10509.0
Sodium: min = 0.0, max = 49.4
Water: min = 0.0, max = 1875.9
Vitamin A: min = 0.0, max = 362.7
Vitamin B1: min = 0.0, max = 25.0
Vitamin B11: min = 0.0, max = 177.6
Vitamin B12: min = 0.0, max = 25.0
Vitamin B2: min = 0.0, max = 35.0
Vitamin B3: min = 0.0, max = 124.0
Vitamin B5: min = 0.0, max = 74.3
Vitamin B6: min = 0.0, max = 15.8
Vitamin C: min = 0.0, max = 3872.0
Vitamin D: min = 0.0, max = 217.6
Vitamin E: min = 0.0, max = 41.6
Vitamin K: min = 0.0, max = 166.4
Calcium: min = 0.0, max = 1283.5
Copper: min = 0.0, max = 1890.0
Iron: min = 0.0, max = 121.2
Magnesium: min = 0.0, max = 921.6
Manganese: min = 0.0, max = 451.0
Phosphorus: min = 0.0, max = 5490.0
Potassium: min = 0.0, max = 11336.9
Selenium: min = 0.0, max = 3308.0
Zinc: min = 0.0, max = 147.3
Nutrition Density: min = 0.0, max = 3911.4

Recipes dataset numeric ranges:
cooking_time_minutes: min = 5, max = 720
prep_time_minutes: min = 2, max = 60
servings: min = 1.0, max = 24.0
calories_per_serving: min = 40.0, max = 784.0
```

Data Cleaning & Sanity Checks

In [12]:
```
# ========================================
# DATA CLEANING & SANITY CHECKS
# ========================================

# ------------------------------------------------
# 1. Remove unnecessary columns in food dataset
# ------------------------------------------------
food_df = food_df.drop(columns=["Unnamed: 0", "Unnamed: 0.1"], errors="ignore")

# ------------------------------------------------
# 2. Standardize column names
# ------------------------------------------------
df.columns = df.columns.str.lower()
food_df.columns = food_df.columns.str.lower().str.replace(" ", "_")
recipes_df.columns = recipes_df.columns.str.lower().str.replace(" ", "_")

# ------------------------------------------------
# 3. Check missing values again
# ------------------------------------------------
print("Missing values after column cleanup:\n")

print("Merged calorie dataset:")
print(df.isnull().sum())

print("\nFood nutrition dataset:")
print(food_df.isnull().sum())

print("\nRecipes dataset:")
print(recipes_df.isnull().sum())

# ------------------------------------------------
# 4. Handle missing values in recipes dataset
# ------------------------------------------------

# servings and calories_per_serving have few missing values
recipes_df["servings"] = recipes_df["servings"].fillna(recipes_df["servings"].media
recipes_df["calories_per_serving"] = recipes_df["calories_per_serving"].fillna(
    recipes_df["calories_per_serving"].median()
)

# ------------------------------------------------
# 5. Clean dietary restriction column
# ------------------------------------------------
recipes_df["dietary_restrictions"] = recipes_df["dietary_restrictions"].replace("['

# ------------------------------------------------
# 6. Sanity check: remove unrealistic calorie values
# ------------------------------------------------
df = df[(df["calories"] > 0) & (df["calories"] < 1000)]

# ------------------------------------------------
# 7. Sanity check: remove unrealistic durations
# ------------------------------------------------
df = df[(df["duration"] > 0) & (df["duration"] <= 60)]
```

```python
# ------------------------------------------------
# 8. Feature Engineering (important for ML)
# ------------------------------------------------

# BMI
df["bmi"] = df["weight"] / ((df["height"]/100) ** 2)

# calories per minute
df["calories_per_min"] = df["calories"] / df["duration"]

# ------------------------------------------------
# 9. Clean food dataset column names
# ------------------------------------------------
food_df = food_df.rename(columns={
    "caloric_value": "calories",
    "dietary_fiber": "fiber"
})

# ------------------------------------------------
# 10. Select key nutrition features for recommendation
# ------------------------------------------------
nutrition_df = food_df[
    [
        "food",
        "calories",
        "protein",
        "fat",
        "carbohydrates",
        "sugars",
        "fiber",
        "sodium",
        "nutrition_density"
    ]
]

# ------------------------------------------------
# 11. Final dataset previews
# ------------------------------------------------
print("\nCleaned calorie dataset:")
display(df.head())

print("\nCleaned nutrition dataset:")
display(nutrition_df.head())

print("\nCleaned recipes dataset:")
display(recipes_df.head())

# ------------------------------------------------
# 12. Final shapes
# ------------------------------------------------
print("Final calorie dataset shape:", df.shape)
print("Final nutrition dataset shape:", nutrition_df.shape)
print("Final recipes dataset shape:", recipes_df.shape)
```

```
Missing values after column cleanup:

Merged calorie dataset:
user_id       0
gender        0
age           0
height        0
weight        0
duration      0
heart_rate    0
body_temp     0
calories      0
dtype: int64

Food nutrition dataset:
food                    0
caloric_value           0
fat                     0
saturated_fats          0
monounsaturated_fats    0
polyunsaturated_fats    0
carbohydrates           0
sugars                  0
protein                 0
dietary_fiber           0
cholesterol             0
sodium                  0
water                   0
vitamin_a               0
vitamin_b1              0
vitamin_b11             0
vitamin_b12             0
vitamin_b2              0
vitamin_b3              0
vitamin_b5              0
vitamin_b6              0
vitamin_c               0
vitamin_d               0
vitamin_e               0
vitamin_k               0
calcium                 0
copper                  0
iron                    0
magnesium               0
manganese               0
phosphorus              0
potassium               0
selenium                0
zinc                    0
nutrition_density       0
dtype: int64

Recipes dataset:
recipe_name             0
cuisine                 0
ingredients             0
```

```
cooking_time_minutes    0
prep_time_minutes       0
servings                4
calories_per_serving    5
dietary_restrictions    0
dtype: int64

Cleaned calorie dataset:
```

	user_id	gender	age	height	weight	duration	heart_rate	body_temp	calories	
0	14733363	male	68	190	94	29	105	40.8	231	26.038
1	14861698	female	20	166	60	14	94	40.3	66	21.773
2	11179863	male	69	179	79	5	88	38.7	26	24.655
3	16180408	female	34	179	71	13	100	40.5	71	22.159
4	17771927	female	27	154	58	10	81	39.8	35	24.456

```
Cleaned nutrition dataset:
```

	food	calories	protein	fat	carbohydrates	sugars	fiber	sodium	nutrition_density
0	cream cheese	51	0.9	5.0	0.8	0.500	0.0	0.016	7.070
1	neufchatel cheese	215	7.8	19.4	3.1	2.700	0.0	0.300	130.100
2	requeijao cremoso light catupiry	49	0.8	3.6	0.9	3.400	0.1	0.000	5.400
3	ricotta cheese	30	1.5	2.0	1.5	0.091	0.0	0.017	5.196
4	cream cheese low fat	30	1.2	2.3	1.2	0.900	0.0	0.046	27.007

```
Cleaned recipes dataset:
```

	recipe_name	cuisine	ingredients	cooking_time_minutes	prep_time_minutes	servings	ca
0	Spicy Kimchi Fried Rice	Korean	['Cooked rice', 'Kimchi', 'Gochujang', 'Soy sa...	25	15	2.0	
1	Classic Margherita Pizza	Italian	['Pizza dough', 'Tomato sauce', 'Fresh mozzare...	20	30	4.0	
2	Coconut Chickpea Curry	Indian	['Chickpeas', 'Coconut milk', 'Onion', 'Ginger...	35	15	4.0	
3	Pad See Ew with Tofu	Thai	['Wide rice noodles', 'Tofu', 'Chinese broccol...	20	10	2.0	
4	Beef Bourguignon	French	['Beef chuck', 'Red wine', 'Bacon', 'Onion', '...	150	30	6.0	

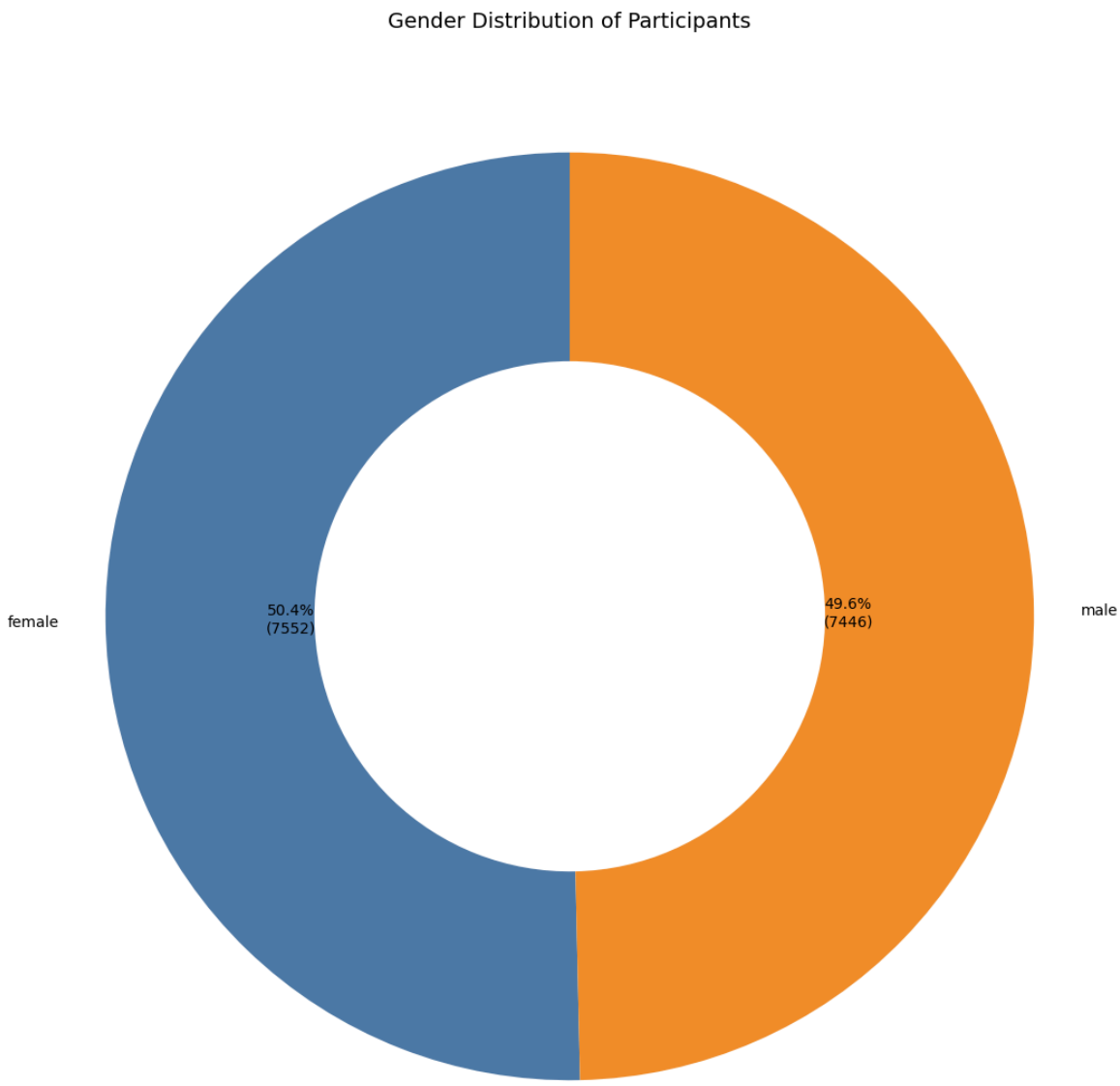
```
Final calorie dataset shape: (15000, 11)
Final nutrition dataset shape: (2395, 9)
Final recipes dataset shape: (161, 8)
```

Key Insights from EDA

1. The dataset contains a balanced distribution of male and female participants.
2. Exercise duration has a strong positive relationship with calories burned.
3. Heart rate also shows a positive association with calorie expenditure.
4. The nutrition dataset contains a wide range of foods with different macronutrient profiles.
5. Several foods have extremely high protein values and may be useful for diet recommendations.
6. The recipes dataset includes diverse cuisines, providing opportunities for meal recommendations.

Gender Distribution

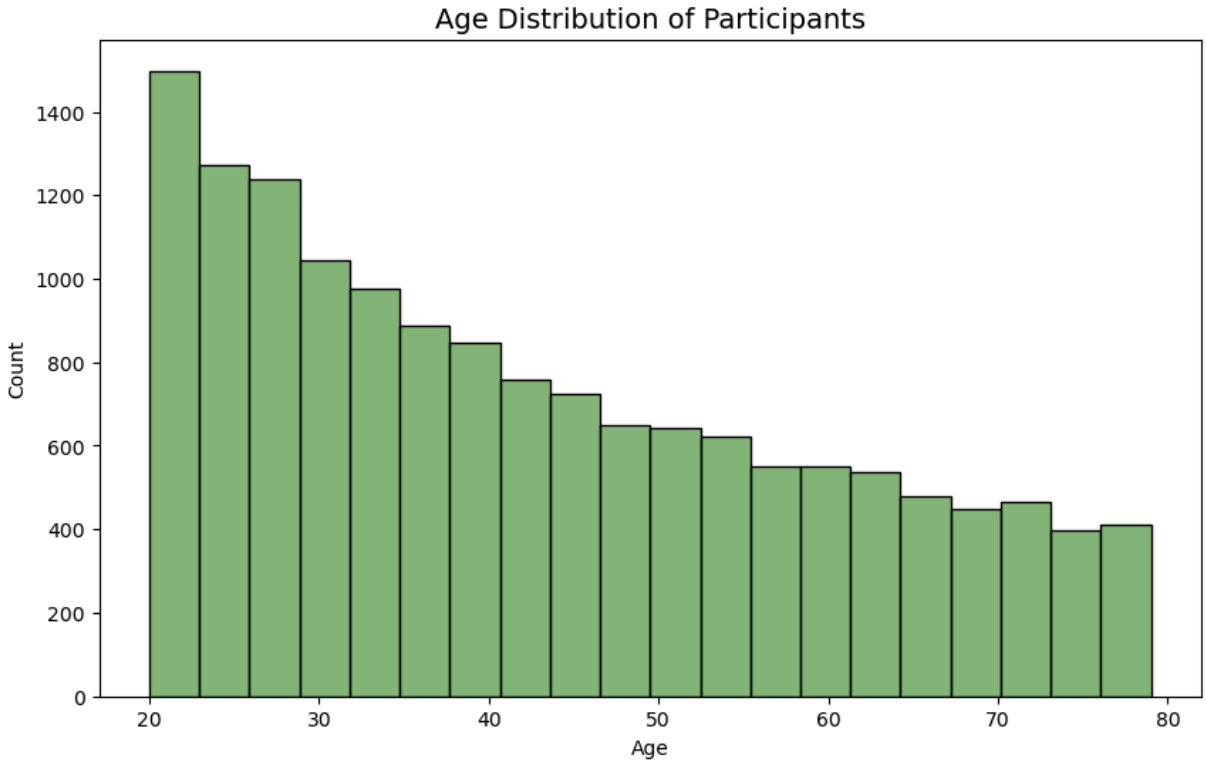
In [28]:
```
plt.figure(figsize=(14,14))

gender_counts = df["gender"].value_counts()

colors = ["#4E79A7", "#F28E2B"]

wedges, texts, autotexts = plt.pie(
    gender_counts,
    labels=gender_counts.index,
    autopct=lambda p: f'{p:.1f}%\n({int(p/100*sum(gender_counts))})',
    colors=colors,
    startangle=90,
    wedgeprops=dict(width=0.45)
)

plt.title("Gender Distribution of Participants", fontsize=14)

plt.show()
```



Age Distribution

In [31]:
```
plt.figure(figsize=(10,6))

sns.histplot(
    df["age"],
    bins=20,
    color="#59A14F",
    edgecolor="black"
)

plt.title("Age Distribution of Participants", fontsize=14)
plt.xlabel("Age")
plt.ylabel("Count")

plt.show()
```

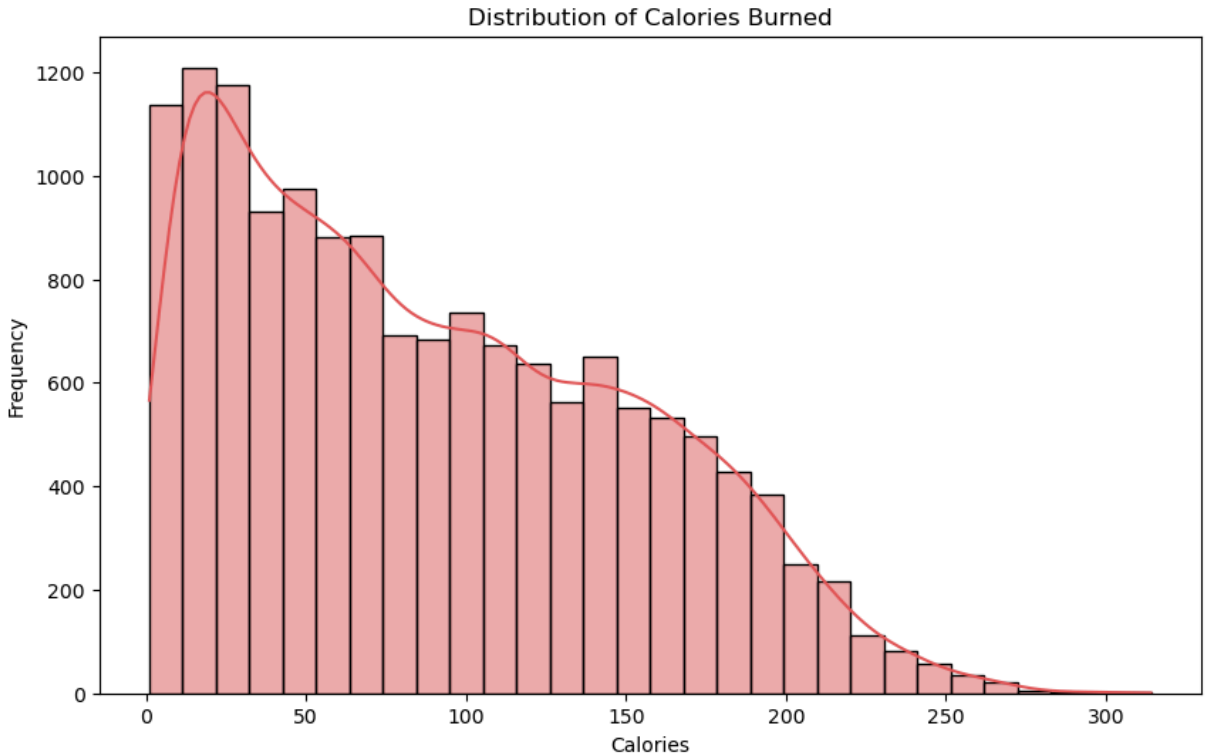
```
print(df["age"].describe())
```




```
count    15000.000000
mean        42.789800
std         16.980264
min         20.000000
25%         28.000000
50%         39.000000
75%         56.000000
max         79.000000
Name: age, dtype: float64
```

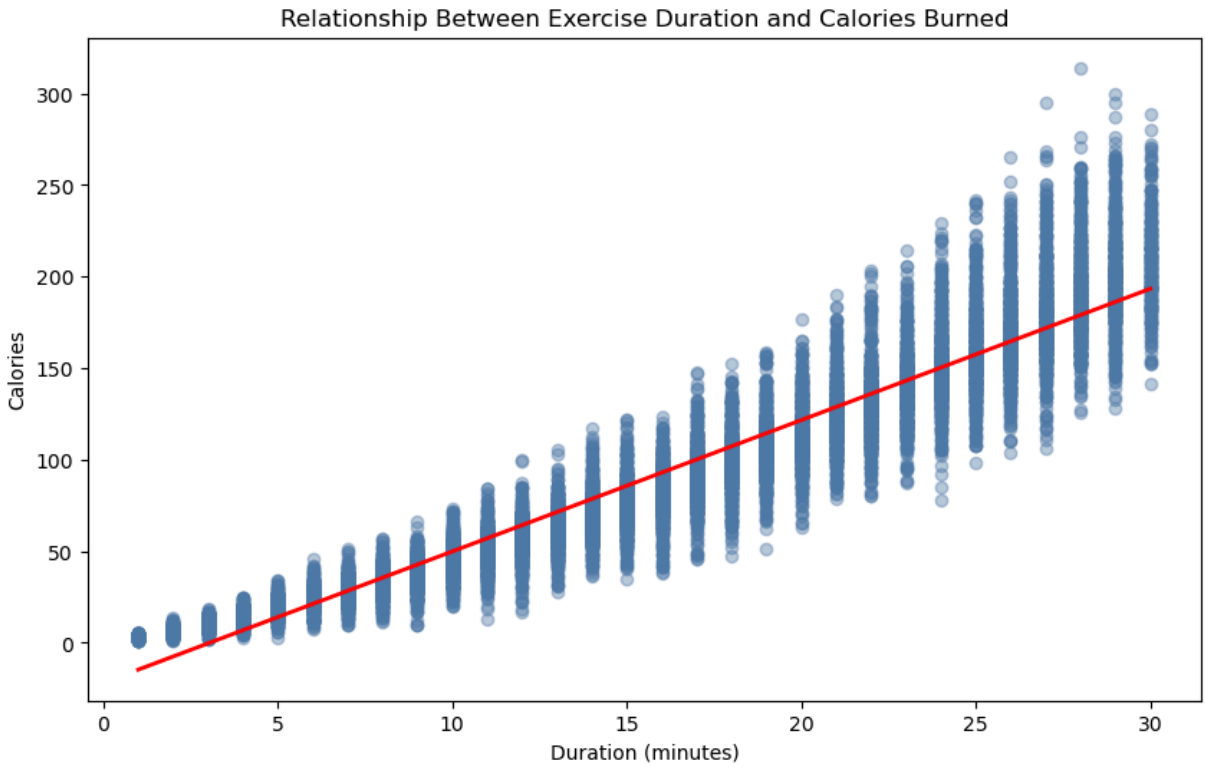
Calories Distribution

In [34]:
```
plt.figure(figsize=(10,6))

sns.histplot(
    df["calories"],
    bins=30,
    kde=True,
    color="#E15759",
    edgecolor="black"
)

plt.title("Distribution of Calories Burned")
plt.xlabel("Calories")
plt.ylabel("Frequency")

plt.show()
```



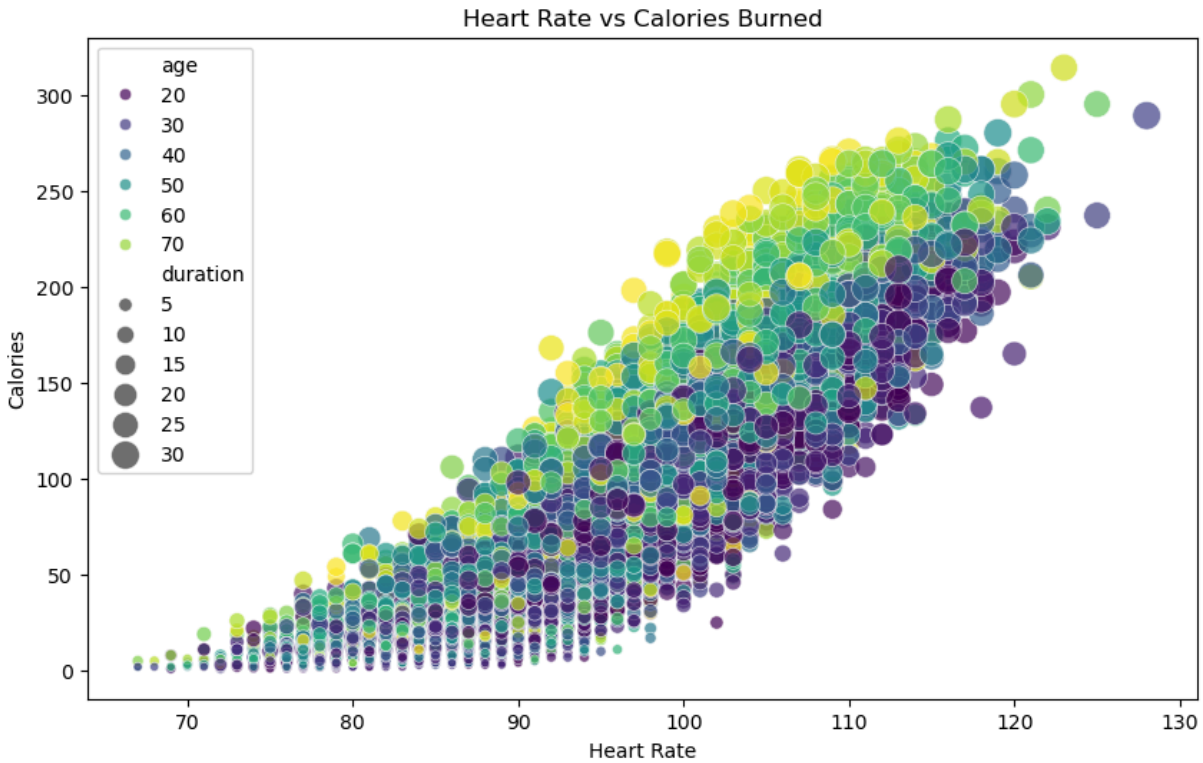
Exercise Duration vs Calories (Scatter + Regression)

In [36]:
```
plt.figure(figsize=(10,6))

sns.regplot(
    data=df,
    x="duration",
    y="calories",
    scatter_kws={"alpha":0.4, "color":"#4E79A7"},
    line_kws={"color":"red", "linewidth":2}
)

plt.title("Relationship Between Exercise Duration and Calories Burned")
plt.xlabel("Duration (minutes)")
plt.ylabel("Calories")

plt.show()
```



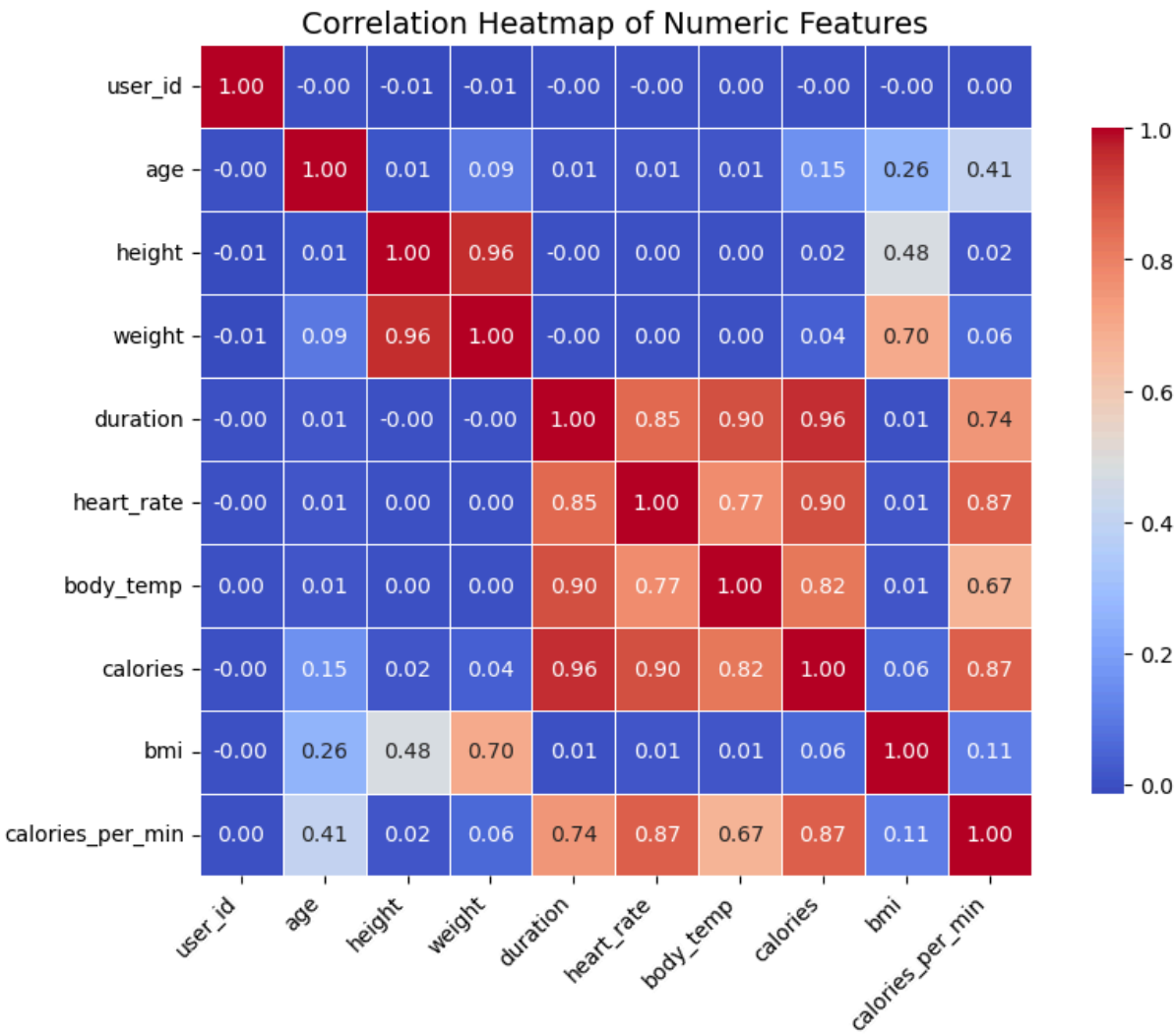
Heart Rate vs Calories Burned

In [39]:
```
plt.figure(figsize=(10,6))

sns.scatterplot(
    data=df,
    x="heart_rate",
    y="calories",
    size="duration",
    sizes=(20,200),
    hue="age",
    palette="viridis",
    alpha=0.7
)

plt.title("Heart Rate vs Calories Burned")
plt.xlabel("Heart Rate")
plt.ylabel("Calories")

plt.show()
```



Correlation Heatmap of Numeric Features

In [56]:
```
plt.figure(figsize=(10,7))

corr = df.select_dtypes(include=["number"]).corr()

sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5,
    square=True,
    cbar_kws={"shrink": 0.8}
)

plt.title("Correlation Heatmap of Numeric Features", fontsize=14)
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.show()
```

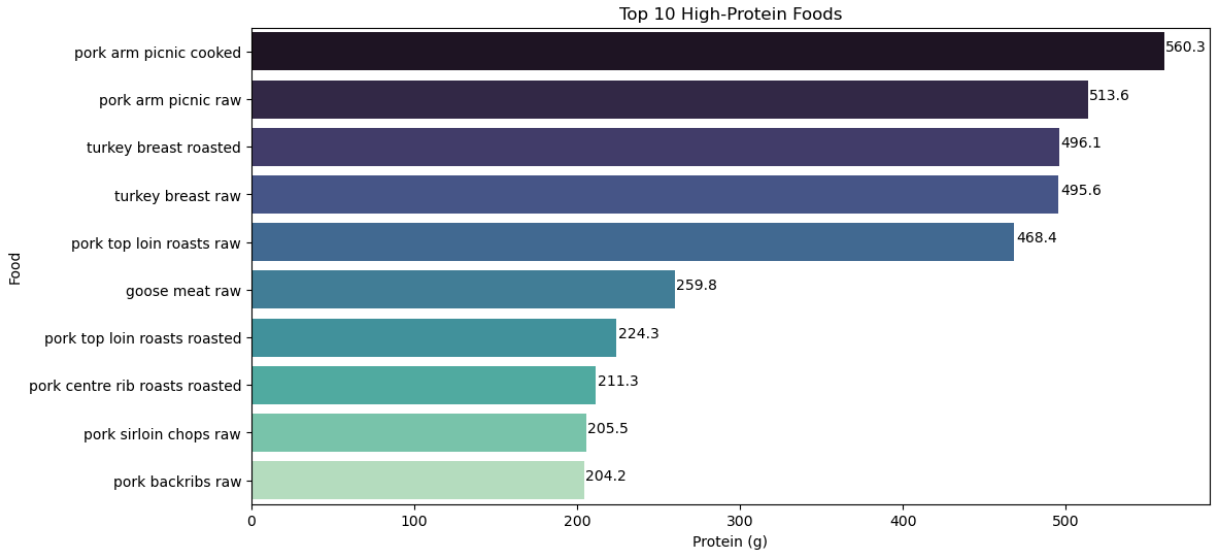


Top 10 High-Protein Foods (Bar Chart with Values)

In [45]:
```
top_protein = nutrition_df.sort_values("protein", ascending=False).head(10)

plt.figure(figsize=(12,6))

bars = sns.barplot(
    data=top_protein,
    x="protein",
    y="food",
    palette="mako"
)

plt.title("Top 10 High-Protein Foods")
plt.xlabel("Protein (g)")
plt.ylabel("Food")

for index, value in enumerate(top_protein["protein"]):
    plt.text(value + 1, index, f"{value:.1f}")
```

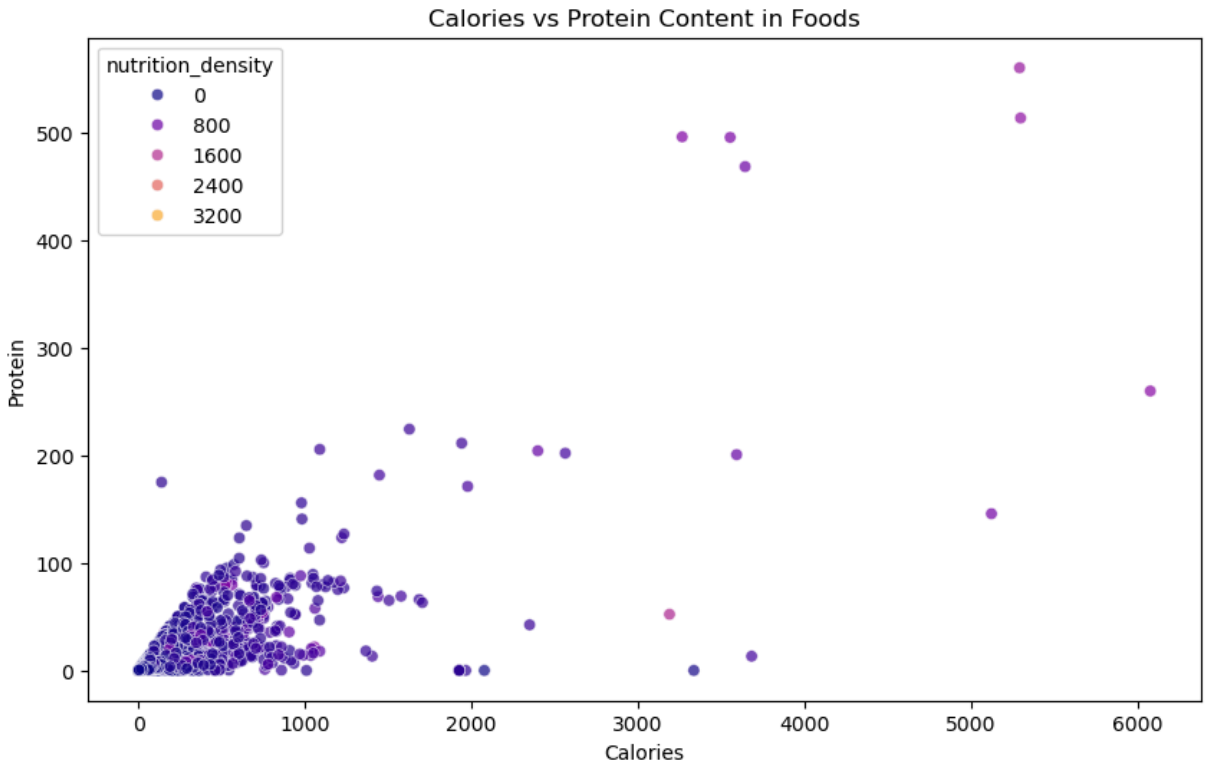
```
plt.show()
```

```
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\3100509853.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  bars = sns.barplot(
```



Calories vs Protein (Nutrition Scatter)

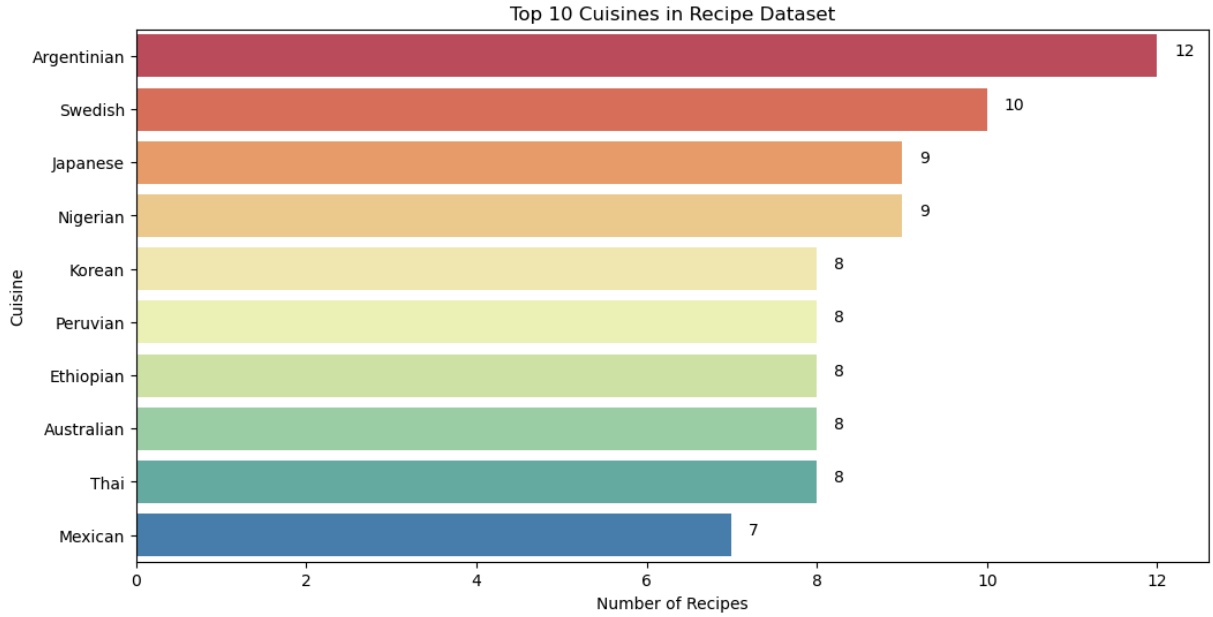
In [47]:
```
plt.figure(figsize=(10,6))

sns.scatterplot(
    data=nutrition_df,
    x="calories",
    y="protein",
    hue="nutrition_density",
    palette="plasma",
    alpha=0.7
)

plt.title("Calories vs Protein Content in Foods")
plt.xlabel("Calories")
plt.ylabel("Protein")

plt.show()
```



Top 10 Cuisines (Bar Chart with Labels)

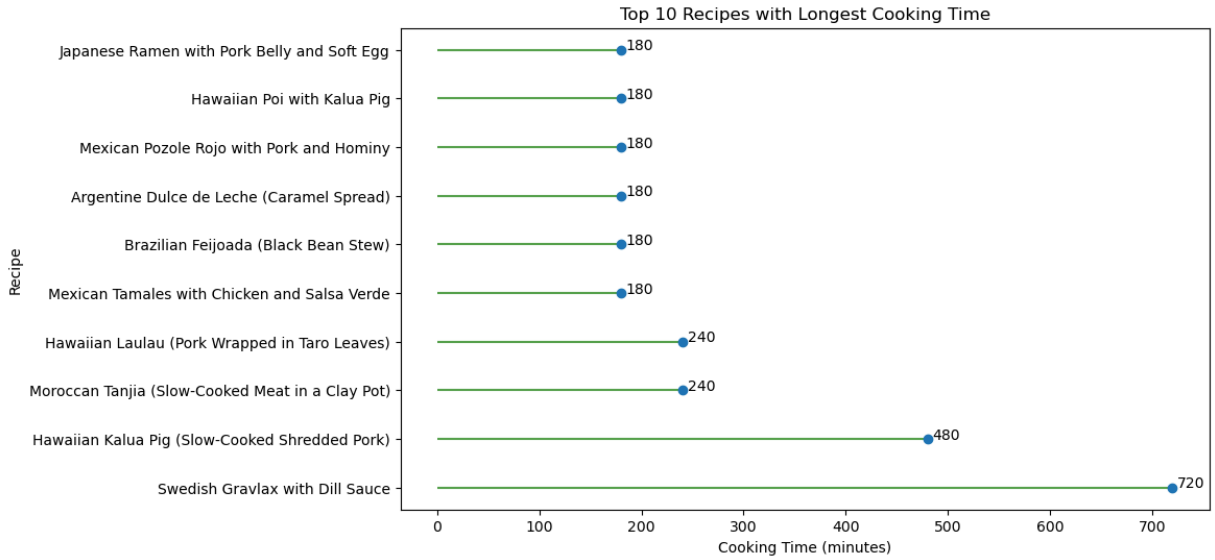
In [51]:
```
cuisine_counts = recipes_df["cuisine"].value_counts().head(10)

plt.figure(figsize=(12,6))

bars = sns.barplot(
    x=cuisine_counts.values,
    y=cuisine_counts.index,
    palette="Spectral"
)

plt.title("Top 10 Cuisines in Recipe Dataset")
plt.xlabel("Number of Recipes")
plt.ylabel("Cuisine")

for index, value in enumerate(cuisine_counts.values):
    plt.text(value + 0.2, index, str(value))

plt.show()
```

```
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\2952791648.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

  bars = sns.barplot(
```



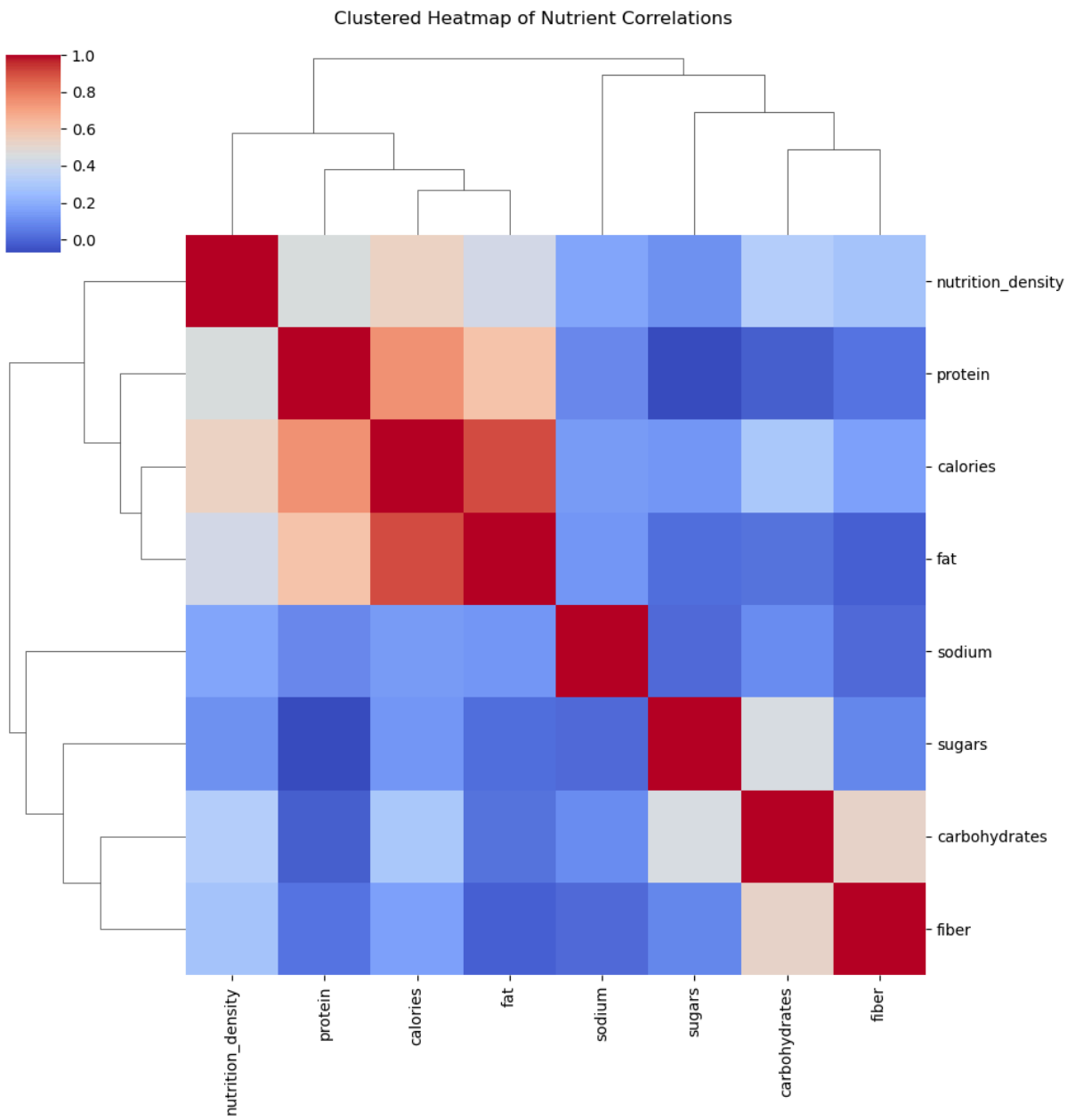
Cooking Time Distribution

In [66]:
```
top_cook = recipes_df.sort_values("cooking_time_minutes", ascending=False).head(10)

plt.figure(figsize=(10,6))

plt.hlines(
    y=top_cook["recipe_name"],
    xmin=0,
    xmax=top_cook["cooking_time_minutes"],
    color="#59A14F"
)

plt.plot(
    top_cook["cooking_time_minutes"],
    top_cook["recipe_name"],
    "o"
)

for i, v in enumerate(top_cook["cooking_time_minutes"]):
    plt.text(v + 5, i, str(v))

plt.title("Top 10 Recipes with Longest Cooking Time")
plt.xlabel("Cooking Time (minutes)")
plt.ylabel("Recipe")

plt.show()
```



Clustered Heatmap of Nutrient Correlations

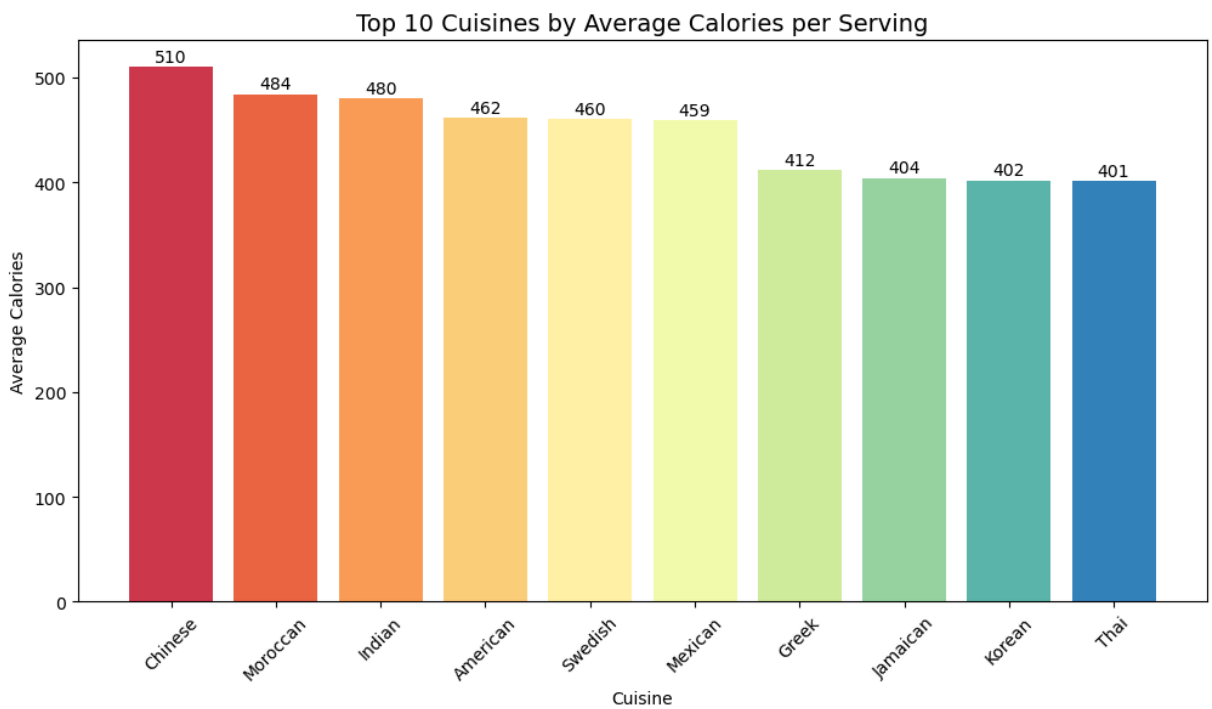
In [74]:
```
nutrition_corr = nutrition_df.drop(columns=["food"]).corr()

sns.clustermap(
    nutrition_corr,
    cmap="coolwarm",
    figsize=(10,10),
    annot=False
)

plt.suptitle("Clustered Heatmap of Nutrient Correlations", y=1.02)

plt.show()
```



Top 10 Cuisines by Average Calories

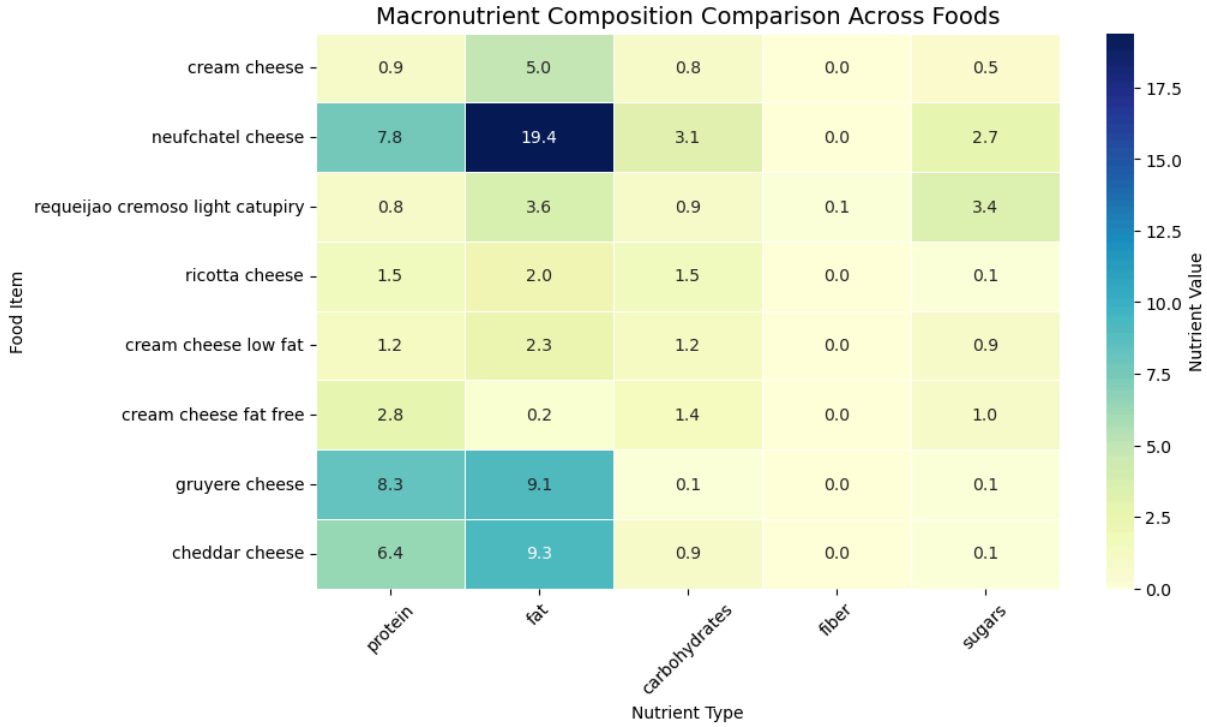
In [76]:
```
avg_calories = recipes_df.groupby("cuisine")["calories_per_serving"].mean().sort_va

plt.figure(figsize=(12,6))

bars = plt.bar(avg_calories.index, avg_calories.values, color=sns.color_palette("Sp

plt.title("Top 10 Cuisines by Average Calories per Serving", fontsize=14)
plt.xlabel("Cuisine")
plt.ylabel("Average Calories")

plt.xticks(rotation=45)

for i,v in enumerate(avg_calories.values):
    plt.text(i, v+5, f"{v:.0f}", ha="center")
```

```
plt.show()
```



Macronutrient Composition Comparison Across Foods

In [78]:
```
plt.figure(figsize=(10,6))

sample_foods = nutrition_df.head(8)

nutrients = ["protein", "fat", "carbohydrates", "fiber", "sugars"]

heatmap_data = sample_foods.set_index("food")[nutrients]

sns.heatmap(
    heatmap_data,
    annot=True,
    fmt=".1f",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={"label":"Nutrient Value"}
)

plt.title("Macronutrient Composition Comparison Across Foods", fontsize=14)
plt.xlabel("Nutrient Type")
plt.ylabel("Food Item")

plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.show()
```

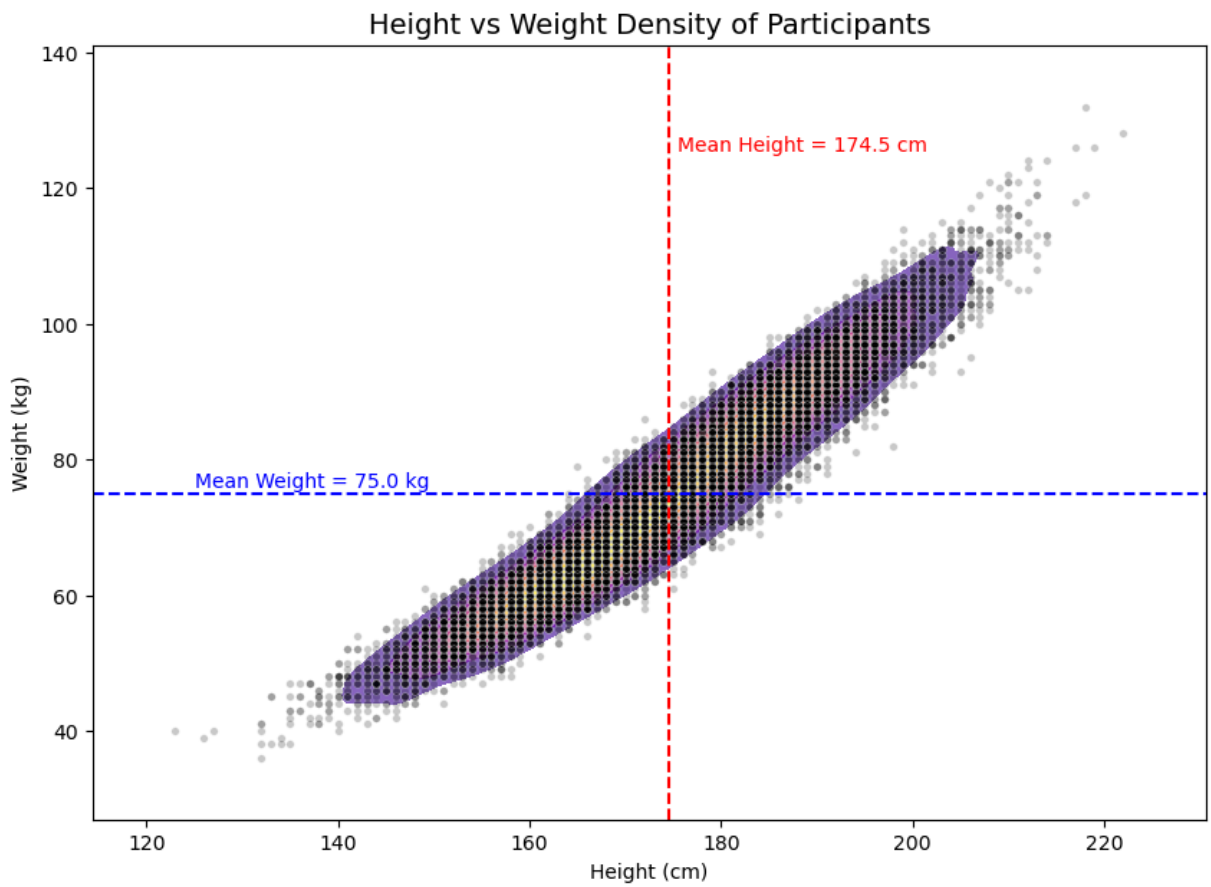


In [ ]:

In [83]:

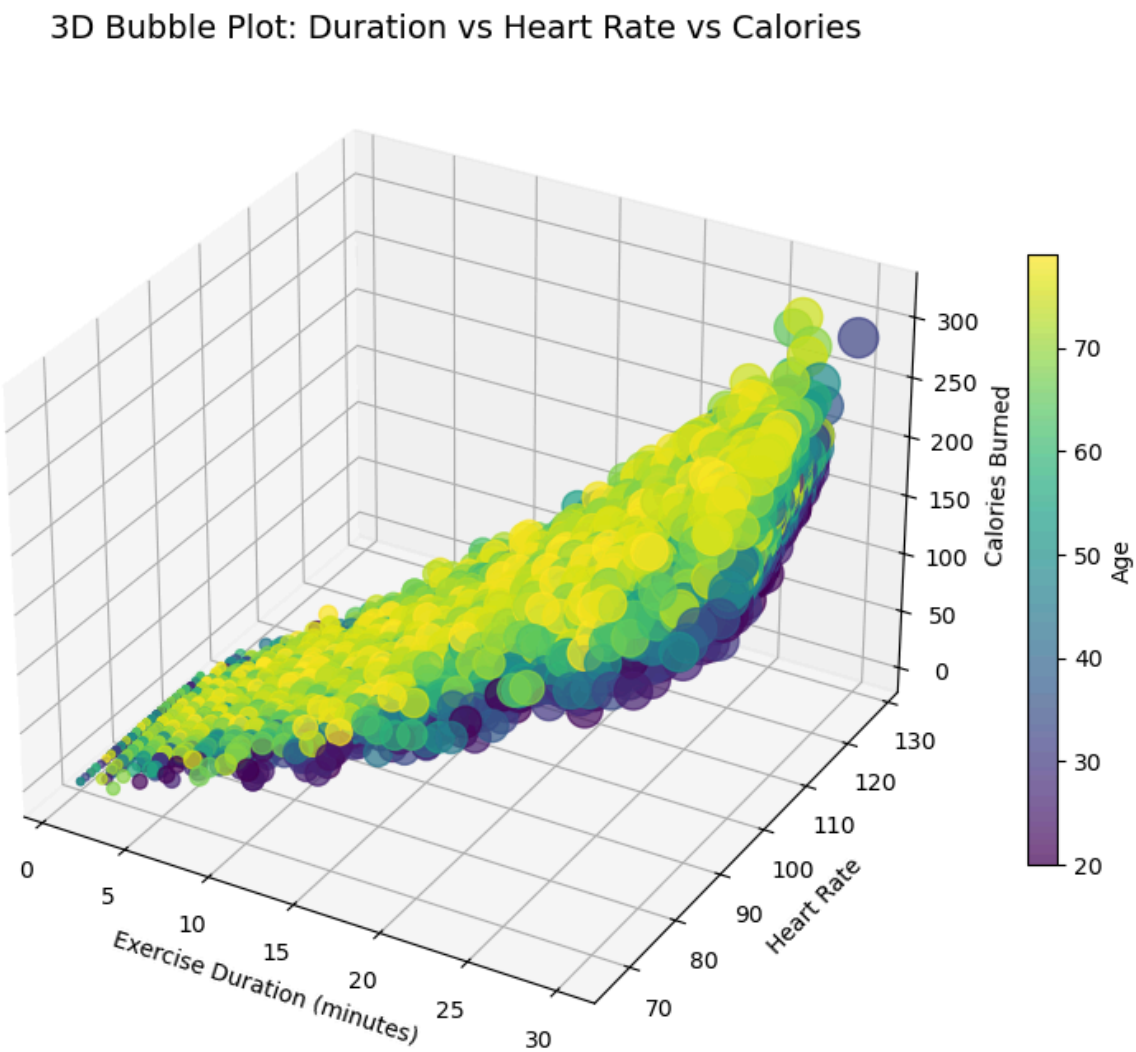
```
plt.figure(figsize=(10,7))

# Density contour
sns.kdeplot(
    data=df,
    x="height",
    y="weight",
    cmap="plasma",
    fill=True,
    thresh=0.05,
    levels=10,
    alpha=0.6
)

# Scatter points
sns.scatterplot(
    data=df,
    x="height",
    y="weight",
    color="black",
    alpha=0.2,
    s=15
)

# Mean values
mean_height = df["height"].mean()
mean_weight = df["weight"].mean()

plt.axvline(mean_height, color="red", linestyle="--")
plt.axhline(mean_weight, color="blue", linestyle="--")
```

```
plt.text(mean_height+1, df["weight"].max()*0.95,
         f"Mean Height = {mean_height:.1f} cm", color="red")

plt.text(df["height"].min()+2, mean_weight+1,
         f"Mean Weight = {mean_weight:.1f} kg", color="blue")

plt.title("Height vs Weight Density of Participants", fontsize=14)
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")

plt.show()
```



In [85]:
```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    df["duration"],
    df["heart_rate"],
    df["calories"],
    s=df["duration"]*10,        # bubble size
    c=df["age"],                # color represents age
    cmap="viridis",
    alpha=0.7
)

# Color bar
cbar = plt.colorbar(scatter, ax=ax, shrink=0.6)
```

```
cbar.set_label("Age")

ax.set_title("3D Bubble Plot: Duration vs Heart Rate vs Calories", fontsize=14)
ax.set_xlabel("Exercise Duration (minutes)")
ax.set_ylabel("Heart Rate")
ax.set_zlabel("Calories Burned")

plt.show()
```



Feature Engineering

Feature engineering is the process of creating new variables from existing data to improve model performance and extract meaningful insights.

In this section, we create additional features for:

1. Calorie prediction dataset
2. Nutrition dataset
3. Recipes dataset

These features will help improve predictive performance and support the nutrition recommendation system.

In [88]:
```
# ========================================
# FEATURE ENGINEERING
# ========================================

# ------------------------------------------------
# 1. Calorie Dataset Feature Engineering
# ------------------------------------------------

# BMI (Body Mass Index)
df["bmi"] = df["weight"] / ((df["height"] / 100) ** 2)

# Calories burned per minute
df["calories_per_min"] = df["calories"] / df["duration"]

# Heart rate intensity
df["heart_rate_intensity"] = df["heart_rate"] / df["age"]

# Exercise efficiency
df["calorie_efficiency"] = df["calories"] / df["heart_rate"]

# ------------------------------------------------
# 2. Nutrition Dataset Feature Engineering
# ------------------------------------------------

# Macronutrient ratios
nutrition_df["protein_ratio"] = nutrition_df["protein"] / nutrition_df["calories"]
nutrition_df["fat_ratio"] = nutrition_df["fat"] / nutrition_df["calories"]
nutrition_df["carb_ratio"] = nutrition_df["carbohydrates"] / nutrition_df["calories

# Sugar percentage
nutrition_df["sugar_ratio"] = nutrition_df["sugars"] / nutrition_df["carbohydrates"

# Fiber density
nutrition_df["fiber_density"] = nutrition_df["fiber"] / nutrition_df["calories"]

# Protein density
nutrition_df["protein_density"] = nutrition_df["protein"] / nutrition_df["calories"

# ------------------------------------------------
# 3. Recipe Dataset Feature Engineering
# ------------------------------------------------

# Total preparation time
recipes_df["total_time"] = (
    recipes_df["cooking_time_minutes"] +
    recipes_df["prep_time_minutes"]
)

# Calories per serving efficiency
recipes_df["calorie_density"] = (
    recipes_df["calories_per_serving"] /
    recipes_df["servings"]
```

```
)

# Ingredient count (approximate complexity)
recipes_df["ingredient_count"] = recipes_df["ingredients"].apply(
    lambda x: len(str(x).split(","))
)

# ----------------------------------------------
# Preview engineered datasets
# ----------------------------------------------

print("Calorie dataset with new features:")
display(df.head())

print("\nNutrition dataset with new features:")
display(nutrition_df.head())

print("\nRecipes dataset with new features:")
display(recipes_df.head())
```

```
Calorie dataset with new features:
```

```
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:26: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["protein_ratio"] = nutrition_df["protein"] / nutrition_df["calories"]
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:27: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["fat_ratio"] = nutrition_df["fat"] / nutrition_df["calories"]
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:28: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["carb_ratio"] = nutrition_df["carbohydrates"] / nutrition_df["calorie
s"]
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:31: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["sugar_ratio"] = nutrition_df["sugars"] / nutrition_df["carbohydrate
s"]
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:34: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["fiber_density"] = nutrition_df["fiber"] / nutrition_df["calories"]
C:\Users\abuha\AppData\Local\Temp\ipykernel_28532\1347443472.py:37: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  nutrition_df["protein_density"] = nutrition_df["protein"] / nutrition_df["calorie
s"]
```

	user_id	gender	age	height	weight	duration	heart_rate	body_temp	calories	
0	14733363	male	68	190	94	29	105	40.8	231	26.038
1	14861698	female	20	166	60	14	94	40.3	66	21.773
2	11179863	male	69	179	79	5	88	38.7	26	24.655
3	16180408	female	34	179	71	13	100	40.5	71	22.159
4	17771927	female	27	154	58	10	81	39.8	35	24.456

```
Nutrition dataset with new features:
```

	food	calories	protein	fat	carbohydrates	sugars	fiber	sodium	nutrition_density
0	cream cheese	51	0.9	5.0	0.8	0.500	0.0	0.016	7.070
1	neufchatel cheese	215	7.8	19.4	3.1	2.700	0.0	0.300	130.100
2	requeijao cremoso light catupiry	49	0.8	3.6	0.9	3.400	0.1	0.000	5.400
3	ricotta cheese	30	1.5	2.0	1.5	0.091	0.0	0.017	5.196
4	cream cheese low fat	30	1.2	2.3	1.2	0.900	0.0	0.046	27.007

```
Recipes dataset with new features:
```

	recipe_name	cuisine	ingredients	cooking_time_minutes	prep_time_minutes	servings	ca
0	Spicy Kimchi Fried Rice	Korean	['Cooked rice', 'Kimchi', 'Gochujang', 'Soy sa...	25	15	2.0	
1	Classic Margherita Pizza	Italian	['Pizza dough', 'Tomato sauce', 'Fresh mozzare...	20	30	4.0	
2	Coconut Chickpea Curry	Indian	['Chickpeas', 'Coconut milk', 'Onion', 'Ginger...	35	15	4.0	
3	Pad See Ew with Tofu	Thai	['Wide rice noodles', 'Tofu', 'Chinese broccol...	20	10	2.0	
4	Beef Bourguignon	French	['Beef chuck', 'Red wine', 'Bacon', 'Onion', '...	150	30	6.0	

In [90]:

```
# ========================================
# Dataset Shapes After Feature Engineering
# ========================================

print("Calorie dataset shape:", df.shape)
print("Nutrition dataset shape:", nutrition_df.shape)
print("Recipes dataset shape:", recipes_df.shape)

print("\nCalorie dataset columns:")
print(df.columns.tolist())

print("\nNutrition dataset columns:")
print(nutrition_df.columns.tolist())

print("\nRecipes dataset columns:")
print(recipes_df.columns.tolist())
```

```
Calorie dataset shape: (15000, 13)
Nutrition dataset shape: (2395, 15)
Recipes dataset shape: (161, 11)

Calorie dataset columns:
['user_id', 'gender', 'age', 'height', 'weight', 'duration', 'heart_rate', 'body_tem
p', 'calories', 'bmi', 'calories_per_min', 'heart_rate_intensity', 'calorie_efficien
cy']

Nutrition dataset columns:
['food', 'calories', 'protein', 'fat', 'carbohydrates', 'sugars', 'fiber', 'sodium',
'nutrition_density', 'protein_ratio', 'fat_ratio', 'carb_ratio', 'sugar_ratio', 'fib
er_density', 'protein_density']

Recipes dataset columns:
['recipe_name', 'cuisine', 'ingredients', 'cooking_time_minutes', 'prep_time_minute
s', 'servings', 'calories_per_serving', 'dietary_restrictions', 'total_time', 'calor
ie_density', 'ingredient_count']
```

Machine Learning Modeling

In this section, we build machine learning models to predict **calories burned** using physiological and exercise-related variables.

The workflow includes:

1. Preparing feature matrix (X) and target variable (y)
2. Encoding categorical variables
3. Splitting the dataset into training and testing sets
4. Training multiple regression models
5. Evaluating models using several performance metrics

Evaluation Metrics Used:

- **MSE (Mean Squared Error)** – measures average squared prediction error
- **RMSE (Root Mean Squared Error)** – square root of MSE, easier to interpret
- **R^2 (Coefficient of Determination)** – proportion of variance explained by the model
- **Adjusted R^2** – adjusted for number of predictors

In [117…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# ------------------------------------------------
# 1. Define target
```

```
# ------------------------------------------------
y = df["calories"]

# ------------------------------------------------
# 2. Define valid features only
# Remove leaked features:
#   - calories_per_min
#   - calorie_efficiency
# Also remove target and ID
# ------------------------------------------------
X = df.drop(columns=[
    "calories",
    "user_id",
    "calories_per_min",
    "calorie_efficiency"
], errors="ignore")

# ------------------------------------------------
# 3. Encode categorical variables
# ------------------------------------------------
X = pd.get_dummies(X, columns=["gender"], drop_first=True)

print("Feature matrix shape:", X.shape)
print("Target shape:", y.shape)

display(X.head())

# ------------------------------------------------
# 4. Train/Test Split
# ------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)

# ------------------------------------------------
# 5. Initialize models
# ------------------------------------------------
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(
        n_estimators=200,
        random_state=42
    ),
    "Gradient Boosting": GradientBoostingRegressor(
        random_state=42
    )
}

# ------------------------------------------------
# 6. Train and evaluate models
```

```
# ------------------------------------------------
results = []

n = X_test.shape[0]
p = X_test.shape[1]

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    results.append({
        "Model": name,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
        "Adj_R2": adj_r2
    })

results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False).reset_inde

print("Corrected Model Comparison Table:")
display(results_df)
```

```
Feature matrix shape: (15000, 9)
Target shape: (15000,)
```

	age	height	weight	duration	heart_rate	body_temp	bmi	heart_rate_intensity	ge
0	68	190	94	29	105	40.8	26.038781	1.544118	
1	20	166	60	14	94	40.3	21.773842	4.700000	
2	69	179	79	5	88	38.7	24.655910	1.275362	
3	34	179	71	13	100	40.5	22.159109	2.941176	
4	27	154	58	10	81	39.8	24.456063	3.000000	

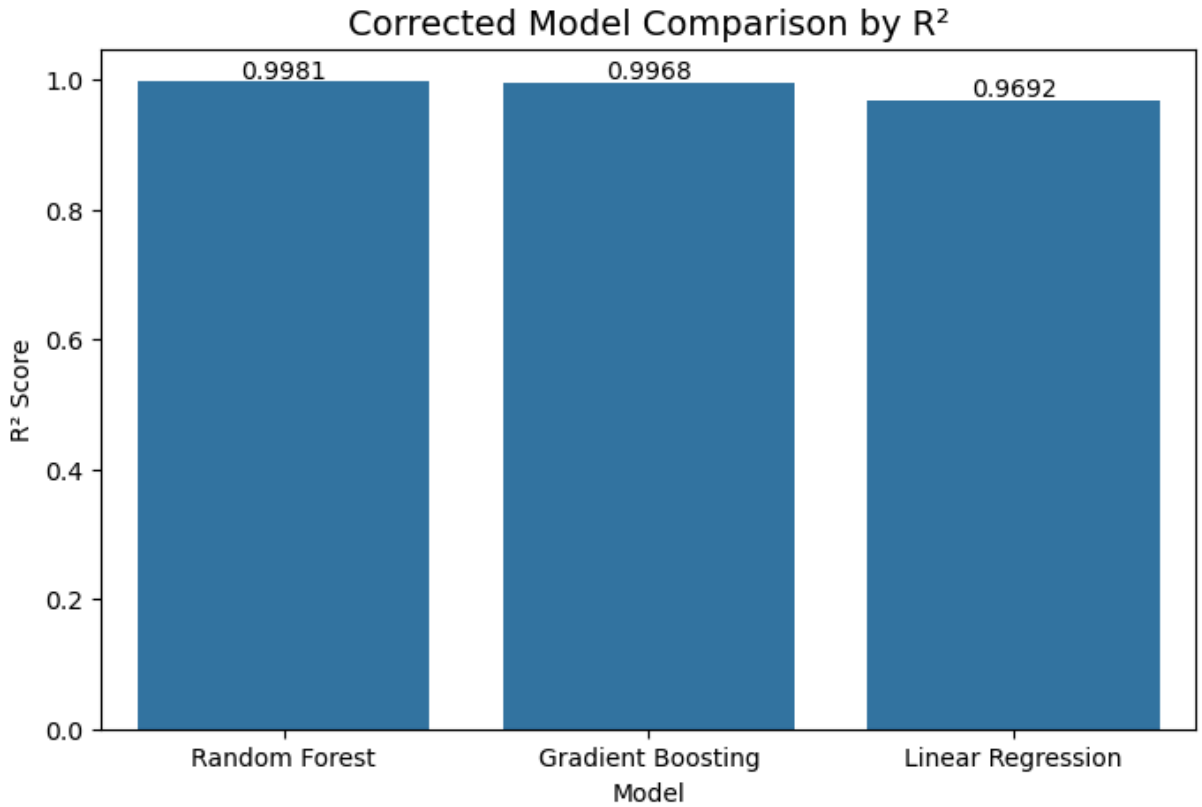
```
Training set shape: (12000, 9)
Test set shape: (3000, 9)
Corrected Model Comparison Table:
```

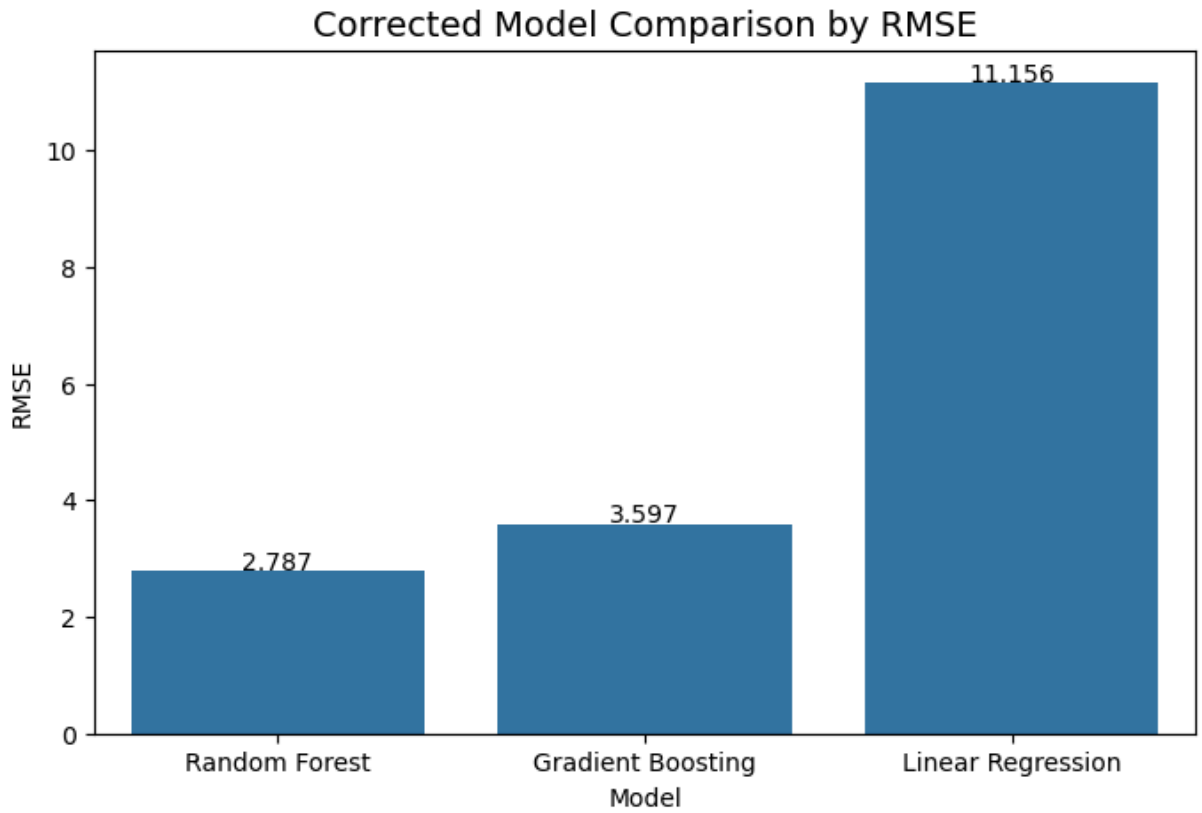
	Model	MSE	RMSE	R2	Adj_R2
0	Random Forest	7.769656	2.787410	0.998075	0.998069
1	Gradient Boosting	12.937651	3.596895	0.996794	0.996785
2	Linear Regression	124.460667	11.156194	0.969161	0.969068

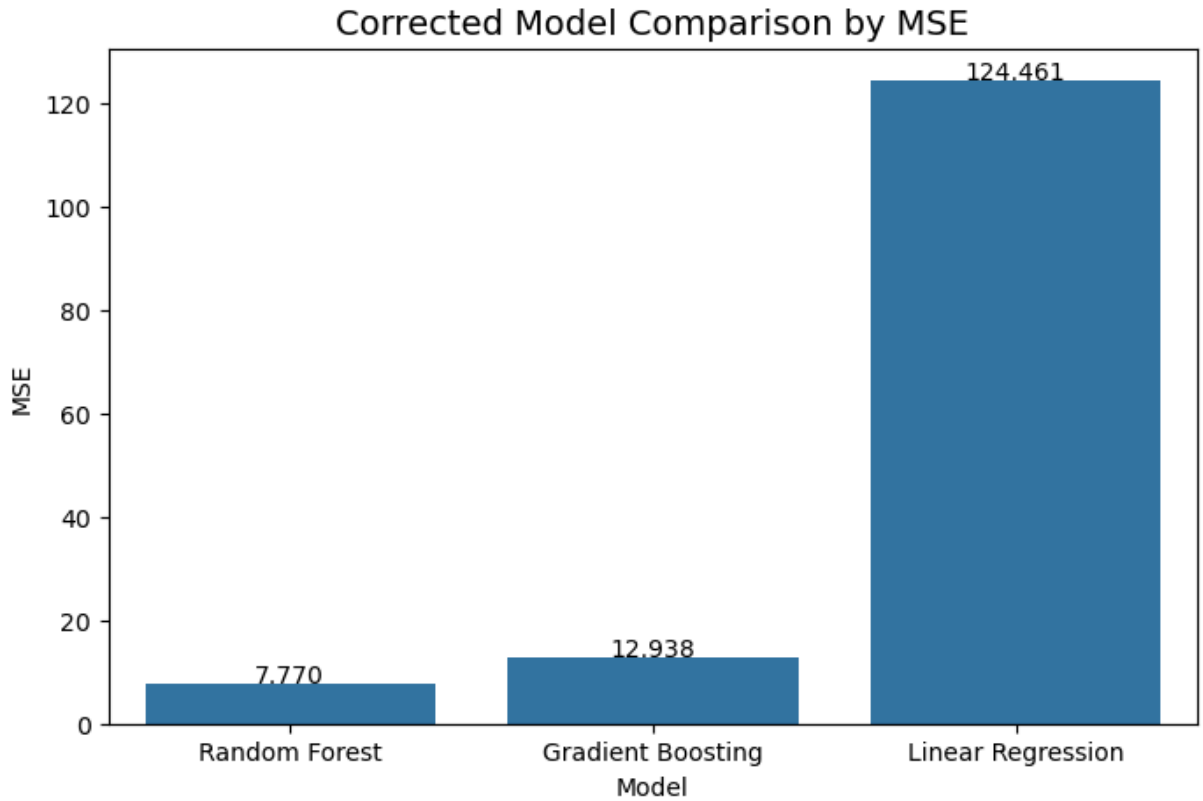
MODEL COMPARISON VISUALIZATIONS

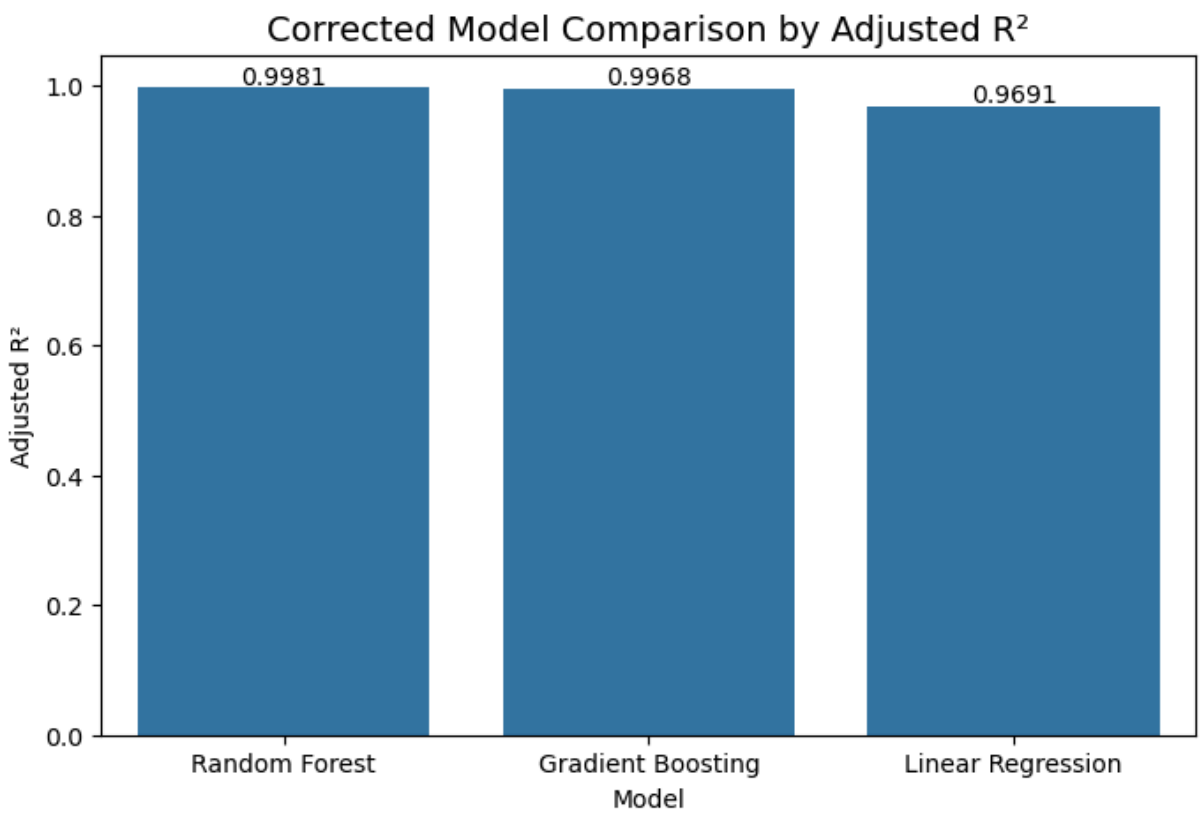
In [120…

```
# R2
plt.figure(figsize=(8,5))
ax = sns.barplot(data=results_df, x="Model", y="R2")
plt.title("Corrected Model Comparison by R²", fontsize=14)
plt.xlabel("Model")
plt.ylabel("R² Score")

for i, v in enumerate(results_df["R2"]):
    ax.text(i, v + 0.005, f"{v:.4f}", ha="center")

plt.show()

# RMSE
plt.figure(figsize=(8,5))
rmse_plot = results_df.sort_values("RMSE", ascending=True)
ax = sns.barplot(data=rmse_plot, x="Model", y="RMSE")
plt.title("Corrected Model Comparison by RMSE", fontsize=14)
plt.xlabel("Model")
plt.ylabel("RMSE")

for i, v in enumerate(rmse_plot["RMSE"]):
    ax.text(i, v + 0.02, f"{v:.3f}", ha="center")

plt.show()

# MSE
plt.figure(figsize=(8,5))
mse_plot = results_df.sort_values("MSE", ascending=True)
ax = sns.barplot(data=mse_plot, x="Model", y="MSE")
plt.title("Corrected Model Comparison by MSE", fontsize=14)
plt.xlabel("Model")
plt.ylabel("MSE")

for i, v in enumerate(mse_plot["MSE"]):
    ax.text(i, v + 0.1, f"{v:.3f}", ha="center")

plt.show()

# Adjusted R2
plt.figure(figsize=(8,5))
adj_plot = results_df.sort_values("Adj_R2", ascending=False)
ax = sns.barplot(data=adj_plot, x="Model", y="Adj_R2")
plt.title("Corrected Model Comparison by Adjusted R²", fontsize=14)
plt.xlabel("Model")
plt.ylabel("Adjusted R²")

for i, v in enumerate(adj_plot["Adj_R2"]):
    ax.text(i, v + 0.005, f"{v:.4f}", ha="center")

plt.show()
```


Corrected Model Comparison by R²
0.9981
0.9968
0.9692
1.0
0.8
0.6
0.4
0.2
0.0
R² Score
Random Forest
Gradient Boosting
Linear Regression
Model


Corrected Model Comparison by RMSE
11.156
3.597
2.787
10
8
6
4
2
0
RMSE
Random Forest
Gradient Boosting
Linear Regression
Model





In [125…

```
# ========================================
# FIT BEST MODEL
# ========================================

best_model_name = results_df.iloc[0]["Model"]
print("Best model selected:", best_model_name)
```

```
if best_model_name == "Random Forest":
    best_model = RandomForestRegressor(n_estimators=200, random_state=42)
elif best_model_name == "Gradient Boosting":
    best_model = GradientBoostingRegressor(random_state=42)
else:
    best_model = LinearRegression()

best_model.fit(X_train, y_train)
```

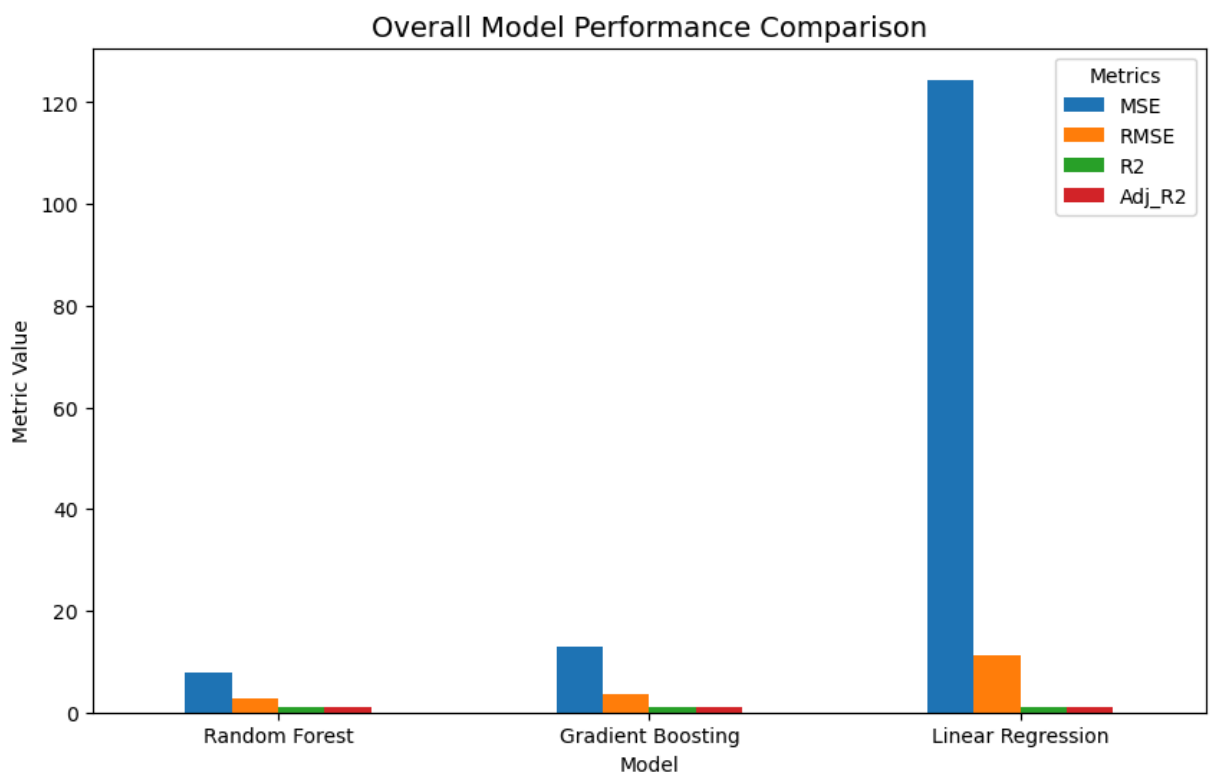
```
Best model selected: Random Forest
```

Out[125…

```
RandomForestRegressor
RandomForestRegressor(n_estimators=200, random_state=42)
```

In [123…

```
plot_df = results_df.copy()
plot_df = plot_df.set_index("Model")[["MSE", "RMSE", "R2", "Adj_R2"]]

plot_df.plot(kind="bar", figsize=(10,6))

plt.title("Overall Model Performance Comparison", fontsize=14)
plt.xlabel("Model")
plt.ylabel("Metric Value")
plt.xticks(rotation=0)
plt.legend(title="Metrics")
plt.show()
```



In [ ]:

Model Comparison Insight

The comparison shows that **Random Forest** achieved the best overall performance, with the **lowest MSE and RMSE** and the **highest R^2 and Adjusted R^2**.
Gradient Boosting also performed very strongly, but slightly below Random Forest.
Linear Regression performed well as a baseline model, but it was clearly less accurate than the tree-based models.

Based on these results, **Random Forest** is selected as the best predictive model for calorie estimation in this project.

Feature Importance and Model Explainability

In this section, the best-performing model is interpreted to understand which features contribute most to calorie prediction.

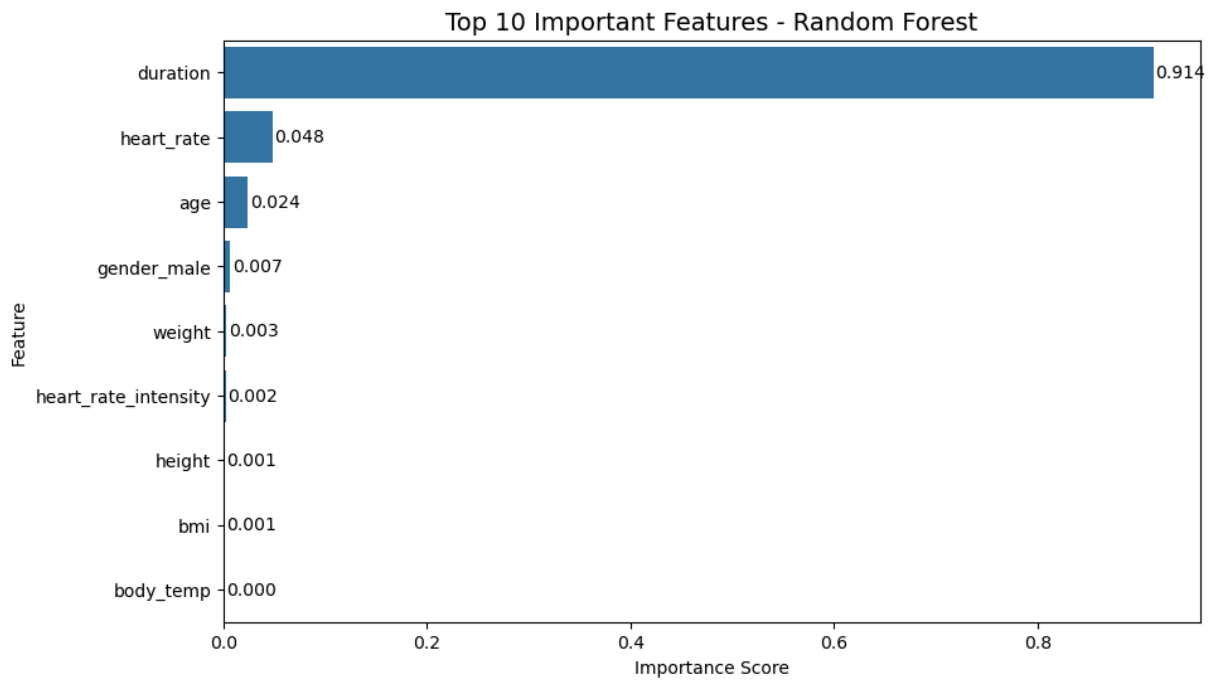
Model explainability is important because it helps identify the key physiological and exercise-related factors influencing calorie expenditure. This also supports responsible AI by making the model more transparent and interpretable.

In [127…

```
# ========================================
# FEATURE IMPORTANCE / EXPLAINABILITY
# ========================================

if hasattr(best_model, "feature_importances_"):
    feature_importance_df = pd.DataFrame({
        "Feature": X_train.columns,
        "Importance": best_model.feature_importances_
    }).sort_values(by="Importance", ascending=False).reset_index(drop=True)

    display(feature_importance_df)

    plt.figure(figsize=(10,6))
    ax = sns.barplot(
        data=feature_importance_df.head(10),
        x="Importance",
        y="Feature"
    )
    plt.title(f"Top 10 Important Features - {best_model_name}", fontsize=14)
    plt.xlabel("Importance Score")
    plt.ylabel("Feature")

    for i, v in enumerate(feature_importance_df.head(10)["Importance"]):
        ax.text(v + 0.002, i, f"{v:.3f}", va="center")

    plt.show()
```

```
else:
    print(f"{best_model_name} does not provide tree-based feature importance.")
```

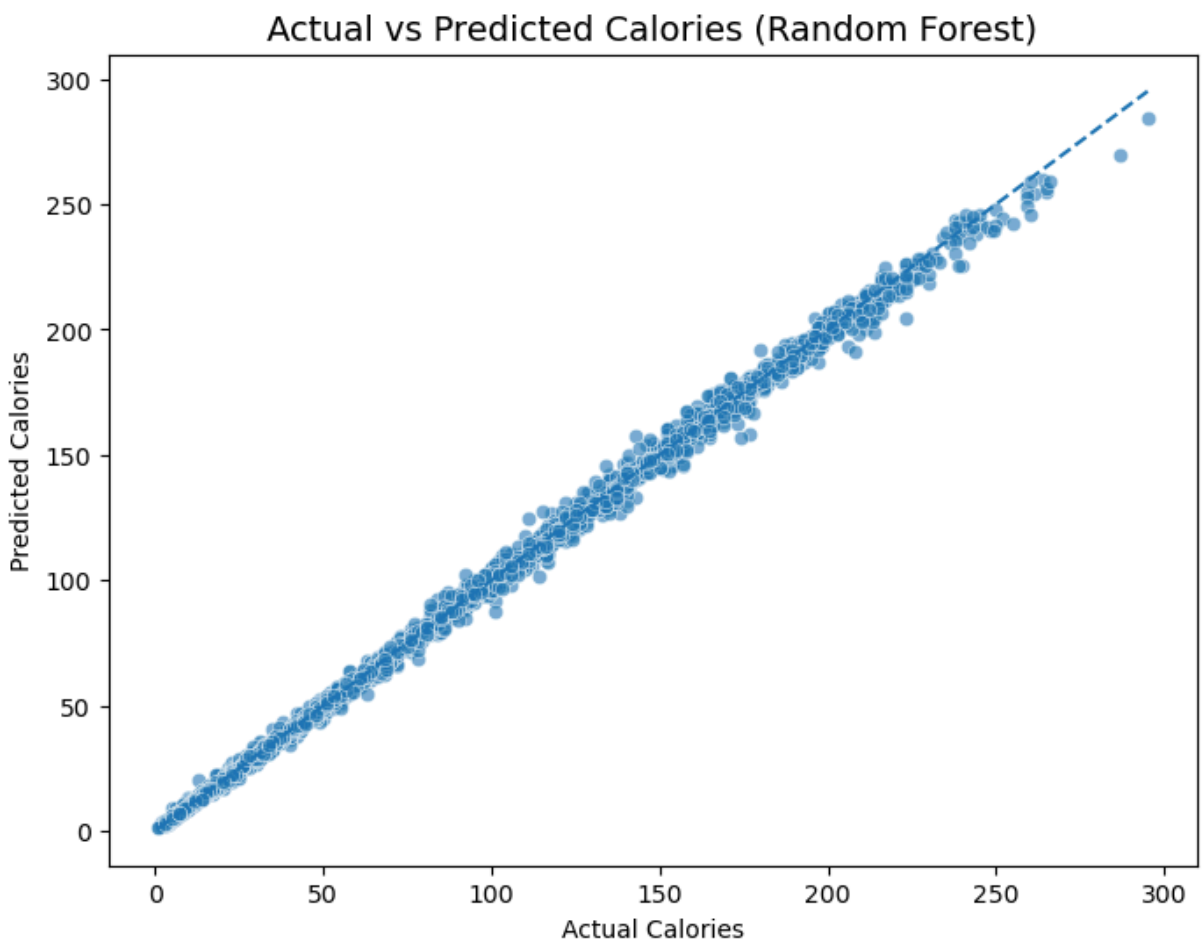
	Feature	Importance
0	duration	0.913887
1	heart_rate	0.047974
2	age	0.024155
3	gender_male	0.006630
4	weight	0.002633
5	heart_rate_intensity	0.002142
6	height	0.001110
7	bmi	0.001057
8	body_temp	0.000413

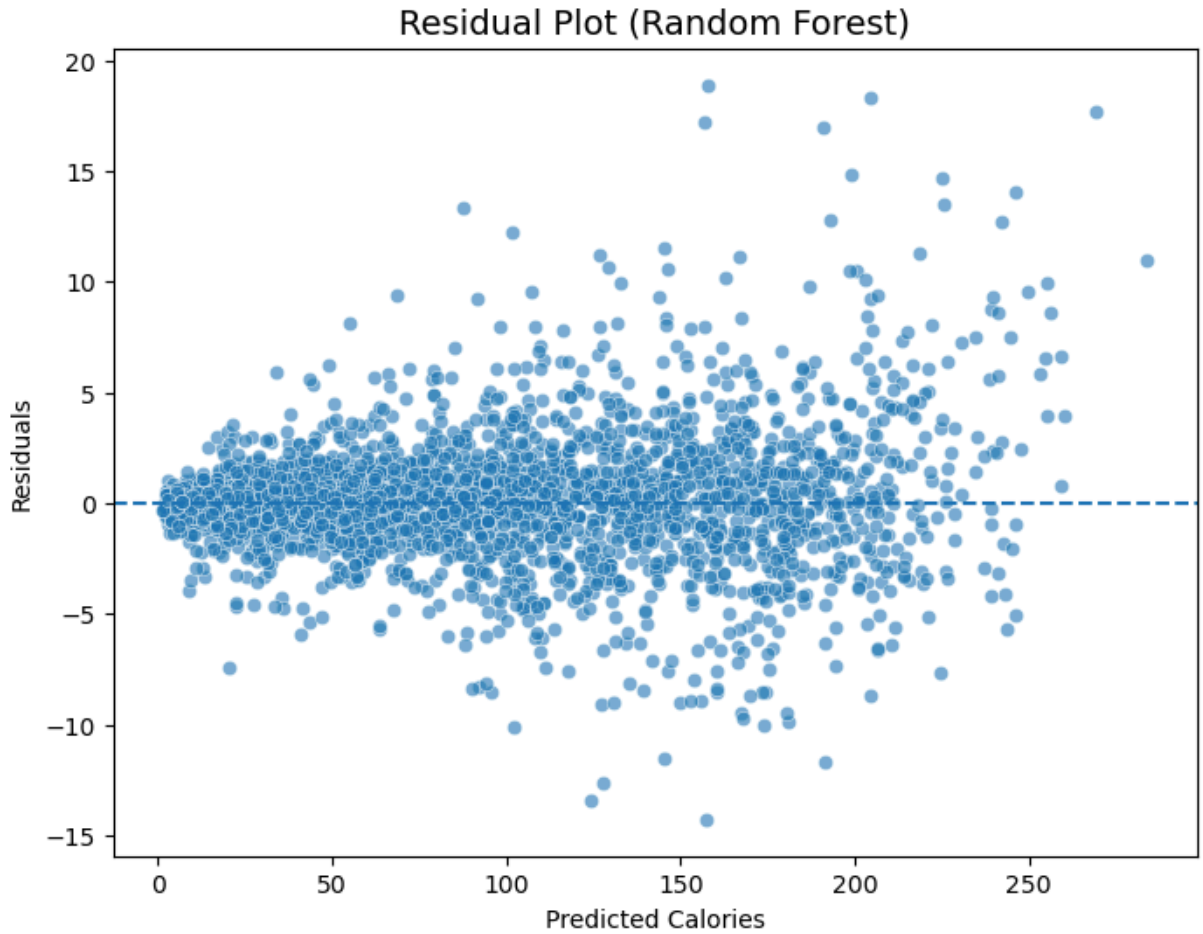


In [129…

```
# ========================================
# PREDICTION DIAGNOSTICS
# ========================================

best_predictions = best_model.predict(X_test)

# Actual vs Predicted
plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=best_predictions, alpha=0.6)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
```

```
)
plt.title(f"Actual vs Predicted Calories ({best_model_name})", fontsize=14)
plt.xlabel("Actual Calories")
plt.ylabel("Predicted Calories")
plt.show()

# Residual plot
residuals = y_test - best_predictions

plt.figure(figsize=(8,6))
sns.scatterplot(x=best_predictions, y=residuals, alpha=0.6)
plt.axhline(y=0, linestyle="--")
plt.title(f"Residual Plot ({best_model_name})", fontsize=14)
plt.xlabel("Predicted Calories")
plt.ylabel("Residuals")
plt.show()
```





Feature Importance Interpretation

The Random Forest model shows that the most important features are those directly related to exercise intensity and body condition, such as duration, heart rate, body temperature, and body composition indicators.

This suggests that calorie expenditure is primarily driven by exercise-related effort and physiological condition rather than demographic variables alone.

The Actual vs Predicted plot shows that the model predictions closely follow the true calorie values, while the residual plot indicates that prediction errors are generally small and centered around zero. This supports the strong performance observed in the evaluation metrics.

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Food Recommendation System

In this section, an AI-powered recommendation system is developed using the trained calorie prediction model and the food nutrition dataset.

The recommendation system works as follows:

1. Predict calorie needs using user physiological and activity features
2. Identify foods that align with the predicted calorie requirement
3. Rank foods based on nutritional quality
4. Recommend recipes that are close to the target calorie range

This step connects predictive modeling with practical nutrition decision support.

In [139…

```
# ----------------------------------------------
# 1. Prepare recommendation datasets
# ----------------------------------------------
recommend_food_df = nutrition_df.copy()
recommend_food_df = recommend_food_df[recommend_food_df["calories"] > 0].copy()
recommend_food_df = recommend_food_df.replace([np.inf, -np.inf], np.nan)

ratio_cols = [
    "protein_ratio", "fat_ratio", "carb_ratio",
    "sugar_ratio", "fiber_density", "protein_density"
]

for col in ratio_cols:
    if col in recommend_food_df.columns:
        recommend_food_df[col] = recommend_food_df[col].fillna(0)

recommend_food_df["recommendation_score"] = (
    0.30 * recommend_food_df["protein_density"] +
    0.25 * recommend_food_df["fiber_density"] +
    0.25 * (recommend_food_df["nutrition_density"] / recommend_food_df["nutrition_d
    0.10 * (recommend_food_df["sugars"] / recommend_food_df["sugars"].max()) -
    0.10 * (recommend_food_df["sodium"] / recommend_food_df["sodium"].max())
)

recommend_recipes_df = recipes_df.copy()

recommend_recipes_df["servings"] = recommend_recipes_df["servings"].fillna(
    recommend_recipes_df["servings"].median()
)
recommend_recipes_df["calories_per_serving"] = recommend_recipes_df["calories_per_s
    recommend_recipes_df["calories_per_serving"].median()
)

if "total_time" not in recommend_recipes_df.columns:
    recommend_recipes_df["total_time"] = (
        recommend_recipes_df["cooking_time_minutes"] +
        recommend_recipes_df["prep_time_minutes"]
```

```
    )

if "ingredient_count" not in recommend_recipes_df.columns:
    recommend_recipes_df["ingredient_count"] = recommend_recipes_df["ingredients"].
        lambda x: len(str(x).split(","))
    )

recommend_recipes_df["recipe_score"] = (
    0.50 * (1 / (recommend_recipes_df["total_time"] + 1)) +
    0.30 * (1 / (recommend_recipes_df["ingredient_count"] + 1)) +
    0.20 * (1 / (recommend_recipes_df["calories_per_serving"] + 1))
)

# ========================================
# USER INPUT PREPARATION
# ========================================

def prepare_user_input(age, gender, height, weight, duration, heart_rate, body_temp
    user_df = pd.DataFrame({
        "age": [age],
        "gender": [gender],
        "height": [height],
        "weight": [weight],
        "duration": [duration],
        "heart_rate": [heart_rate],
        "body_temp": [body_temp]
    })

    # Valid engineered features only
    user_df["bmi"] = user_df["weight"] / ((user_df["height"] / 100) ** 2)
    user_df["heart_rate_intensity"] = user_df["heart_rate"] / user_df["age"]

    # Encode gender
    user_df = pd.get_dummies(user_df, columns=["gender"], drop_first=True)

    # Align with training columns
    user_df = user_df.reindex(columns=X_train.columns, fill_value=0)

    return user_df

# ========================================
# PREDICTION AND RECOMMENDATION FUNCTIONS
# ========================================

def predict_calorie_need(age, gender, height, weight, duration, heart_rate, body_te
    user_input = prepare_user_input(age, gender, height, weight, duration, heart_ra
    predicted_calories = model.predict(user_input)[0]
    return predicted_calories


def recommend_foods(predicted_calories, food_data, top_n=10, tolerance=50):
    filtered_foods = food_data[
        (food_data["calories"] >= predicted_calories - tolerance) &
        (food_data["calories"] <= predicted_calories + tolerance)
    ].copy()
```

```
    if len(filtered_foods) < top_n:
        filtered_foods = food_data[
            (food_data["calories"] >= predicted_calories - 100) &
            (food_data["calories"] <= predicted_calories + 100)
        ].copy()

    filtered_foods = filtered_foods.sort_values(
        by=["recommendation_score", "protein_density", "fiber_density"],
        ascending=False
    )

    return filtered_foods.head(top_n)[[
        "food", "calories", "protein", "fat", "carbohydrates",
        "sugars", "fiber", "sodium", "nutrition_density", "recommendation_score"
    ]]


def recommend_recipes(predicted_calories, recipe_data, top_n=5, tolerance=80):
    filtered_recipes = recipe_data[
        (recipe_data["calories_per_serving"] >= predicted_calories - tolerance) &
        (recipe_data["calories_per_serving"] <= predicted_calories + tolerance)
    ].copy()

    if len(filtered_recipes) < top_n:
        filtered_recipes = recipe_data[
            (recipe_data["calories_per_serving"] >= predicted_calories - 150) &
            (recipe_data["calories_per_serving"] <= predicted_calories + 150)
        ].copy()

    filtered_recipes = filtered_recipes.sort_values(
        by=["recipe_score", "calories_per_serving"],
        ascending=[False, True]
    )

    return filtered_recipes.head(top_n)[[
        "recipe_name", "cuisine", "calories_per_serving",
        "servings", "total_time", "ingredient_count",
        "dietary_restrictions", "recipe_score"
    ]]


def nutrition_recommendation_pipeline(
    age, gender, height, weight, duration, heart_rate, body_temp,
    model, food_data, recipe_data, top_foods=10, top_recipes=5
):
    predicted_calories = predict_calorie_need(
        age=age,
        gender=gender,
        height=height,
        weight=weight,
        duration=duration,
        heart_rate=heart_rate,
        body_temp=body_temp,
        model=model
    )
```

```
    recommended_foods = recommend_foods(
        predicted_calories=predicted_calories,
        food_data=food_data,
        top_n=top_foods
    )

    recommended_recipes = recommend_recipes(
        predicted_calories=predicted_calories,
        recipe_data=recipe_data,
        top_n=top_recipes
    )

    return predicted_calories, recommended_foods, recommended_recipes

# ========================================
# RUN EXAMPLE RECOMMENDATION
# ========================================

example_age = 30
example_gender = "male"
example_height = 175
example_weight = 75
example_duration = 30
example_heart_rate = 100
example_body_temp = 40.0

predicted_calories, recommended_foods, recommended_recipes = nutrition_recommendati
    age=example_age,
    gender=example_gender,
    height=example_height,
    weight=example_weight,
    duration=example_duration,
    heart_rate=example_heart_rate,
    body_temp=example_body_temp,
    model=best_model,
    food_data=recommend_food_df,
    recipe_data=recommend_recipes_df,
    top_foods=10,
    top_recipes=5
)

print(f"Predicted Calories Burned / Calorie Need: {predicted_calories:.2f}")

print("\nTop Recommended Foods:")
display(recommended_foods)

print("\nTop Recommended Recipes:")
display(recommended_recipes)
```

```
Predicted Calories Burned / Calorie Need: 154.13

Top Recommended Foods:
```

	food	calories	protein	fat	carbohydrates	sugars	fiber	sodium	nutrition_
1449	profeel proteiinirahka valio	140	175.0	0.4	19.3	0.2	0.0	0.000	1
37	mozzarella cheese fat free	159	35.8	0.0	4.0	1.7	2.0	0.800	1
2068	tofu raw	188	20.0	11.9	4.7	0.0	0.7	0.060	9
788	wheat bran	125	9.0	2.5	37.4	0.2	24.8	0.015	1
351	walleye pike raw	148	30.4	1.9	0.0	0.0	0.0	0.011	2
386	walleye pike cooked	148	30.4	1.9	0.0	0.0	0.0	0.069	2
606	egg substitute	115	24.0	0.0	4.8	4.8	0.0	0.500	2
505	shrimp canned	128	26.1	1.7	0.0	0.0	0.0	1.100	2
533	lobster raw	116	24.8	1.1	0.0	0.0	0.0	0.600	1
458	ling cooked	168	36.8	1.2	0.0	0.0	0.0	0.300	1

```
Top Recommended Recipes:
```

	recipe_name	cuisine	calories_per_serving	servings	total_time	ingredient_count	d
103	Australian Tim Tam Slam (Chocolate Biscuits wi...	Australian	200.0	1.0	7	2	
125	Argentinian Provoleta (Grilled Provolone Cheese)	Argentinian	200.0	4.0	15	4	
86	Brazilian Brigadeiros (Chocolate Fudge Balls)	Brazilian	100.0	24.0	30	4	
82	Japanese Miso Soup with Tofu and Seaweed	Japanese	80.0	4.0	20	5	
111	Jamaican Bammy (Cassava Flatbread)	Jamaican	120.0	6.0	40	4	

In [ ]: